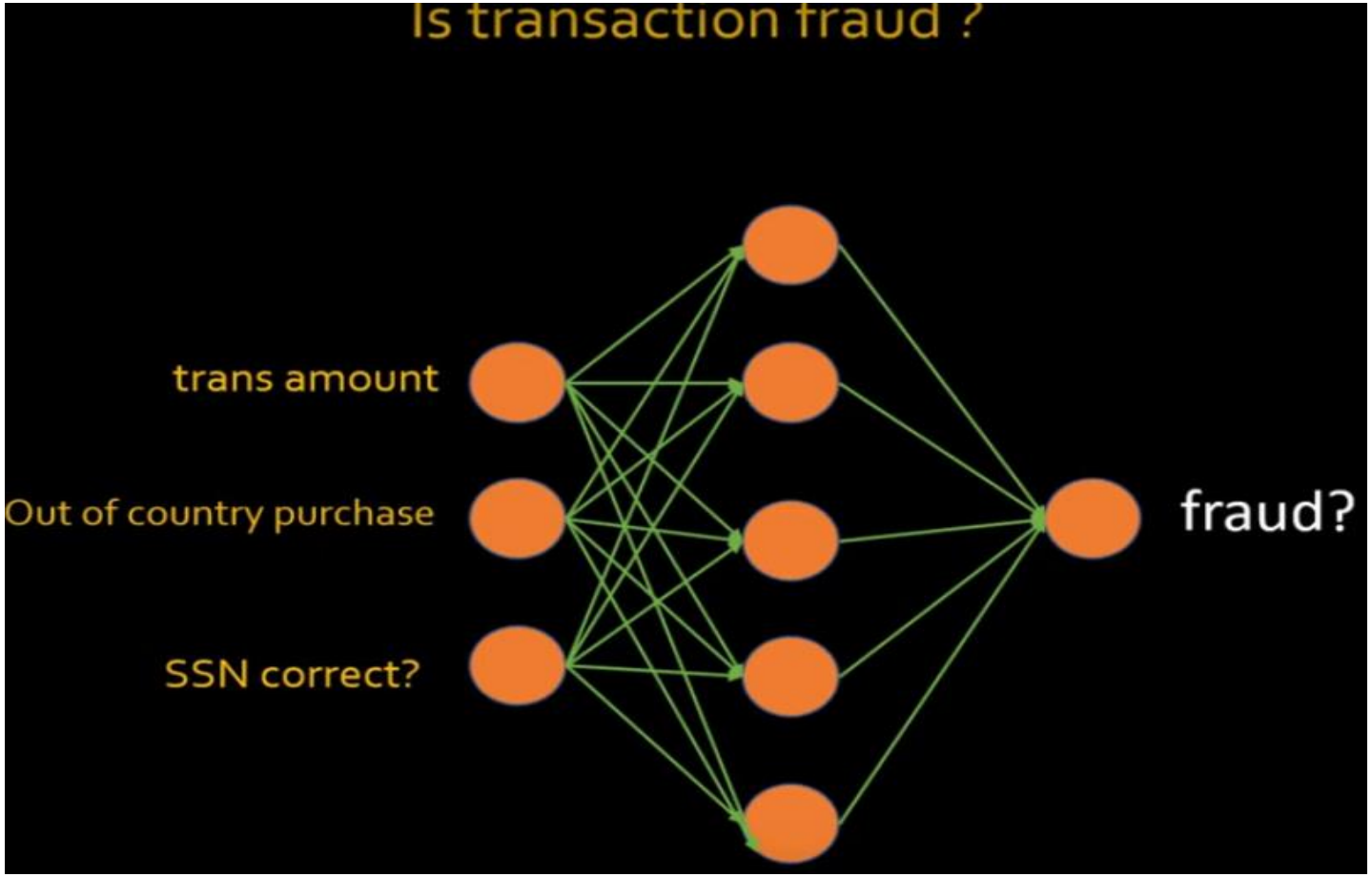
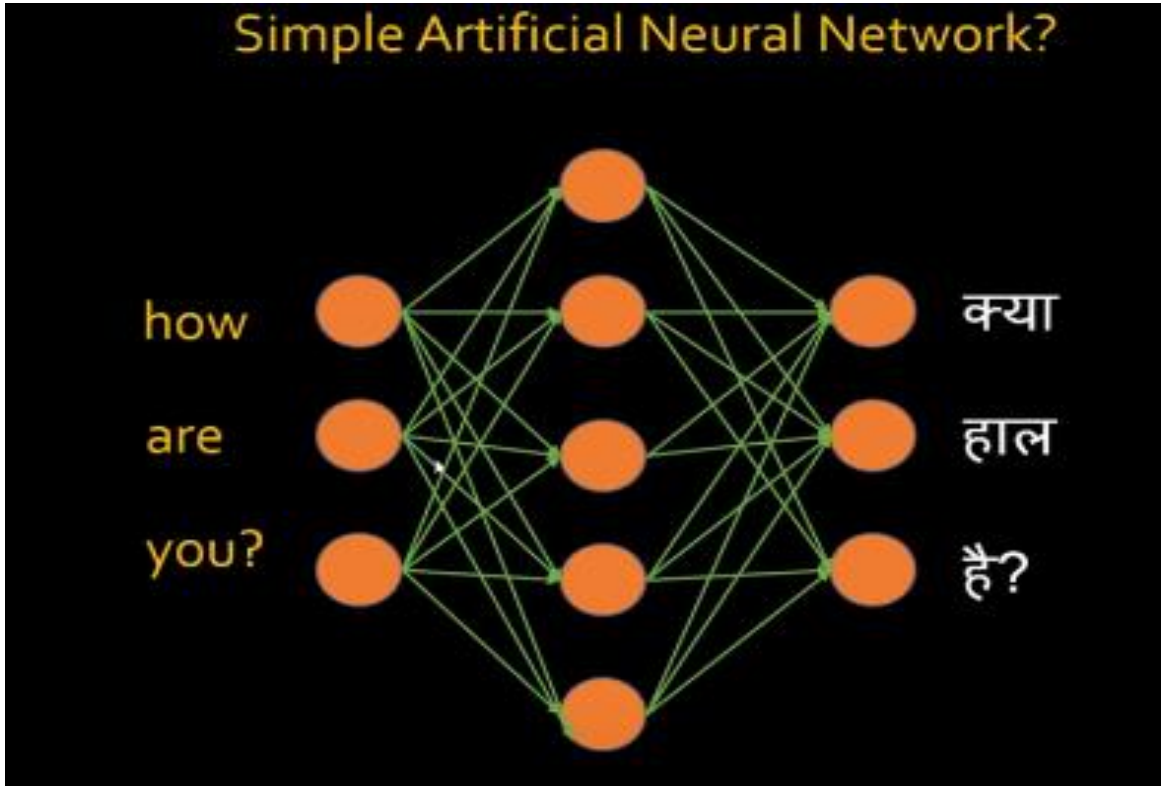
# Module 4

Recurrent Neural Networks(RNN)

# Recurrent Neural Networks(RNN)

- Recurrent neural networks (RNN) are a class of neural networks that are **helpful in modeling sequence data**.
- Sequential data means ordered data in which related things follow each other.
- RNNs  are used mainly in the field of natural language processing as well as some other areas such as speech to text translation, video activity monitoring, etc.
- Examples for sequence data : financial data , DNA sequence.
- RNNs are derived from feedforward networks and  RNNs exhibit similar behavior to how human brains function.
- RNNs are being used in the software behind Google Translate.


Is transaction fraud ?
trans amount
Out of country purchase
SSN correct?
fraud?


Simple Artificial Neural Network?
how
are
you?
क्या
हाल
है?

- RNN’s and feed-forward neural networks get their names from the way they channel information.
- In a feed-forward neural network, the information only moves in one direction — from the input layer, through the hidden layers, to the output layer. The information moves straight through the network and never touches a node twice.

# Why RNNs are required? What is its importance?

- There are a few issues in the feed-forward neural network:

1. Cannot handle sequential data
2. Considers only the current input
3. Cannot memorize previous inputs

- The solution to these issues is the RNN. An RNN can handle sequential data, accepting the current input data, and previously received inputs. RNNs can memorize previous inputs due to their internal memory.

# Advantage of Recurrent Neural Networks(RNN)

- Because of their internal memory, RNN's can remember important things about the input they received, which allows them to be very precise in predicting what's coming next.
- This is why Rnn algorithms are preferred for sequential data like time series, speech, text, financial data, audio, video, weather etc.
- Recurrent neural networks can form a much deeper understanding of a sequence and its context compared to other algorithms.

- Imagine you have a normal feed-forward neural network and give it the word "neuron" as an input and it processes the word character by character.
- By the time it reaches the character "r," it has already forgotten about "n," "e" and "u," which makes it almost impossible for this type of neural network to predict which character would come next.
- A recurrent neural network, however, is able to remember those characters because of its internal memory. It produces output, copies that output and loops it back into the network.

- When ANN is used for sequence modelling problems , some major issues are there.

1. Variable size of input /output neurons.
2. Too much computations
3. No parameter sharing :

Parameter sharing is **sharing of weights by all neurons in a particular feature map**. This helps to reduce the number of parameters in the whole system and makes the computation more efficient.

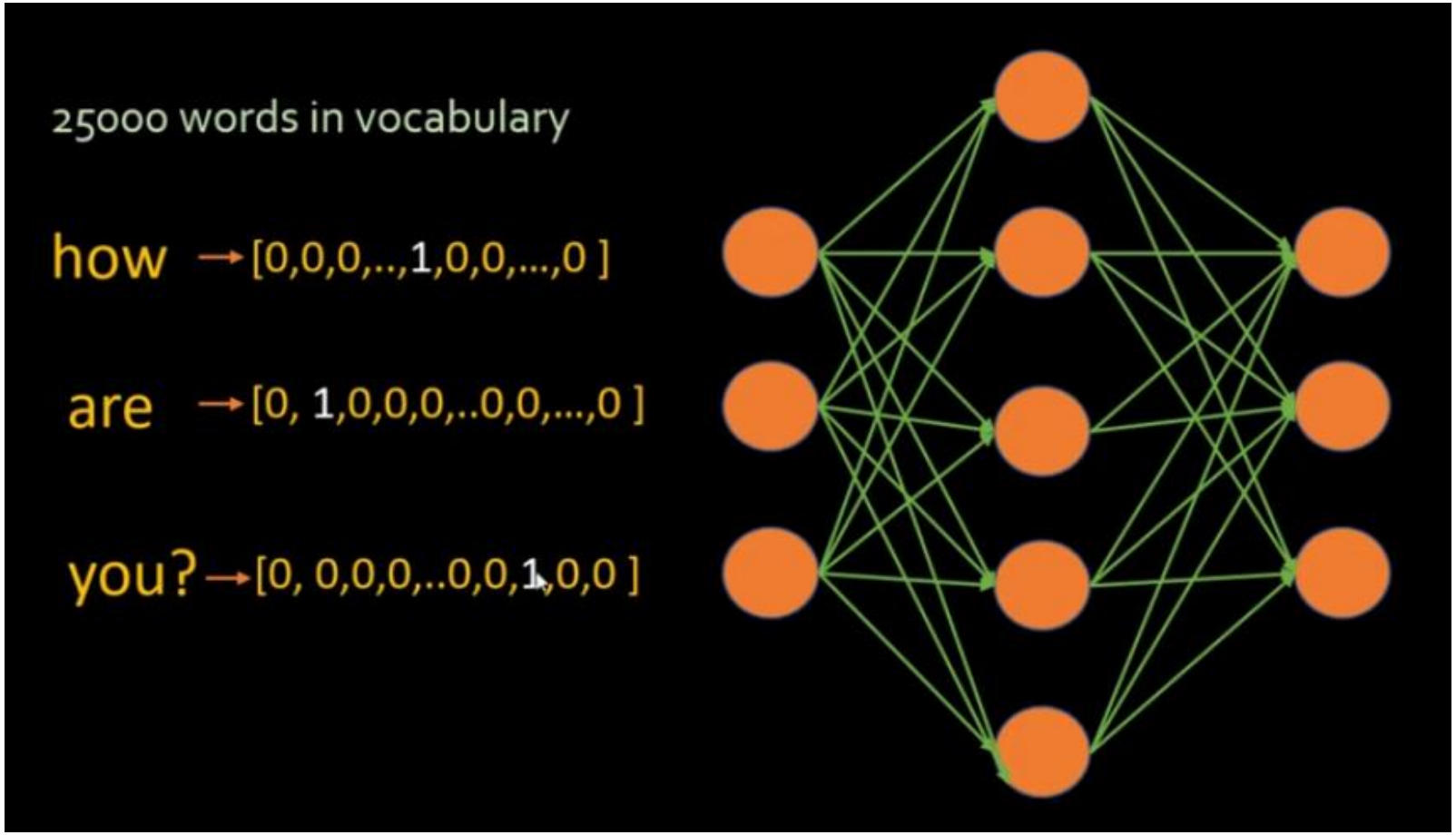
India , Japan , US
(Label encoding)

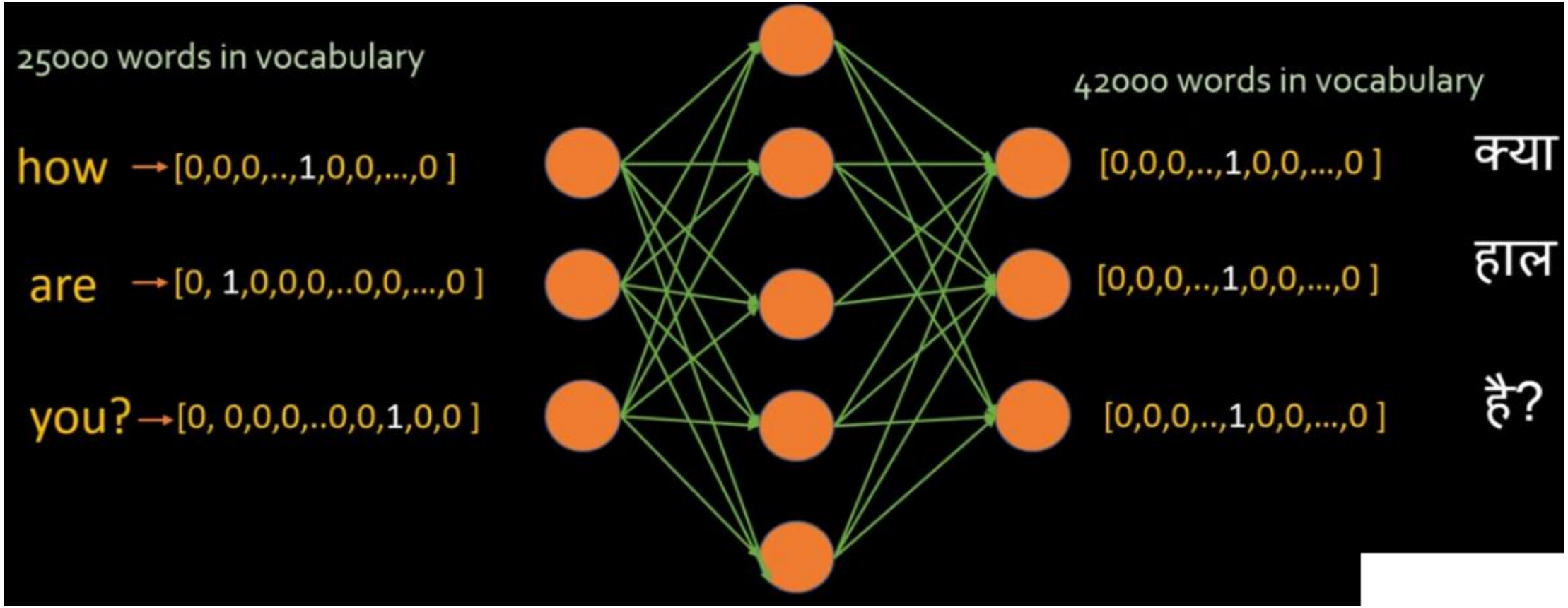
| Country | Age | Salary |
|---|---|---|
| India | 44 | 72000 |
| US | 34 | 65000 |
| Japan | 46 | 98000 |
| US | 35 | 45000 |
| Japan | 23 | 34000 |

| Country | Age | Salary |
|---|---|---|
| 0 | 44 | 72000 |
| 2 | 34 | 65000 |
| 1 | 46 | 98000 |
| 2 | 35 | 45000 |
| 1 | 23 | 34000 |

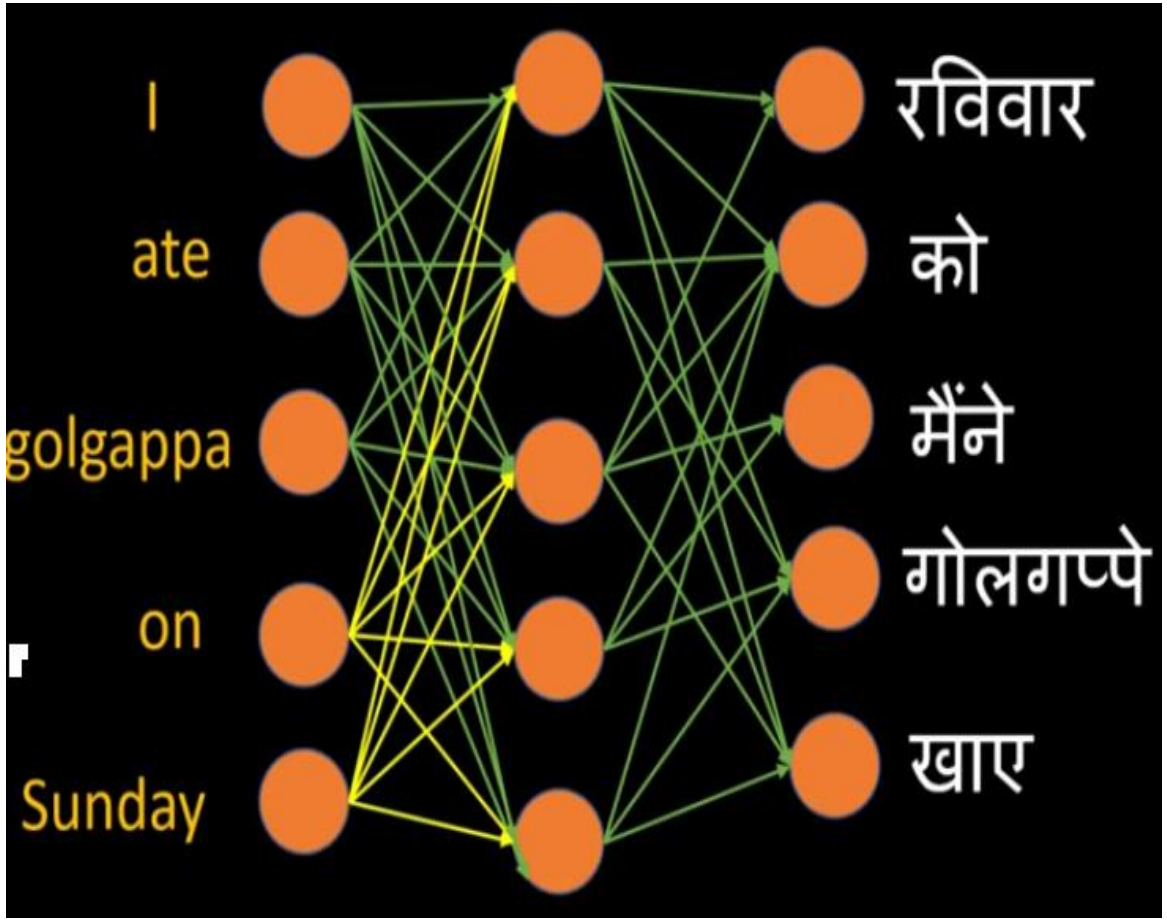
One-Hot Encoding

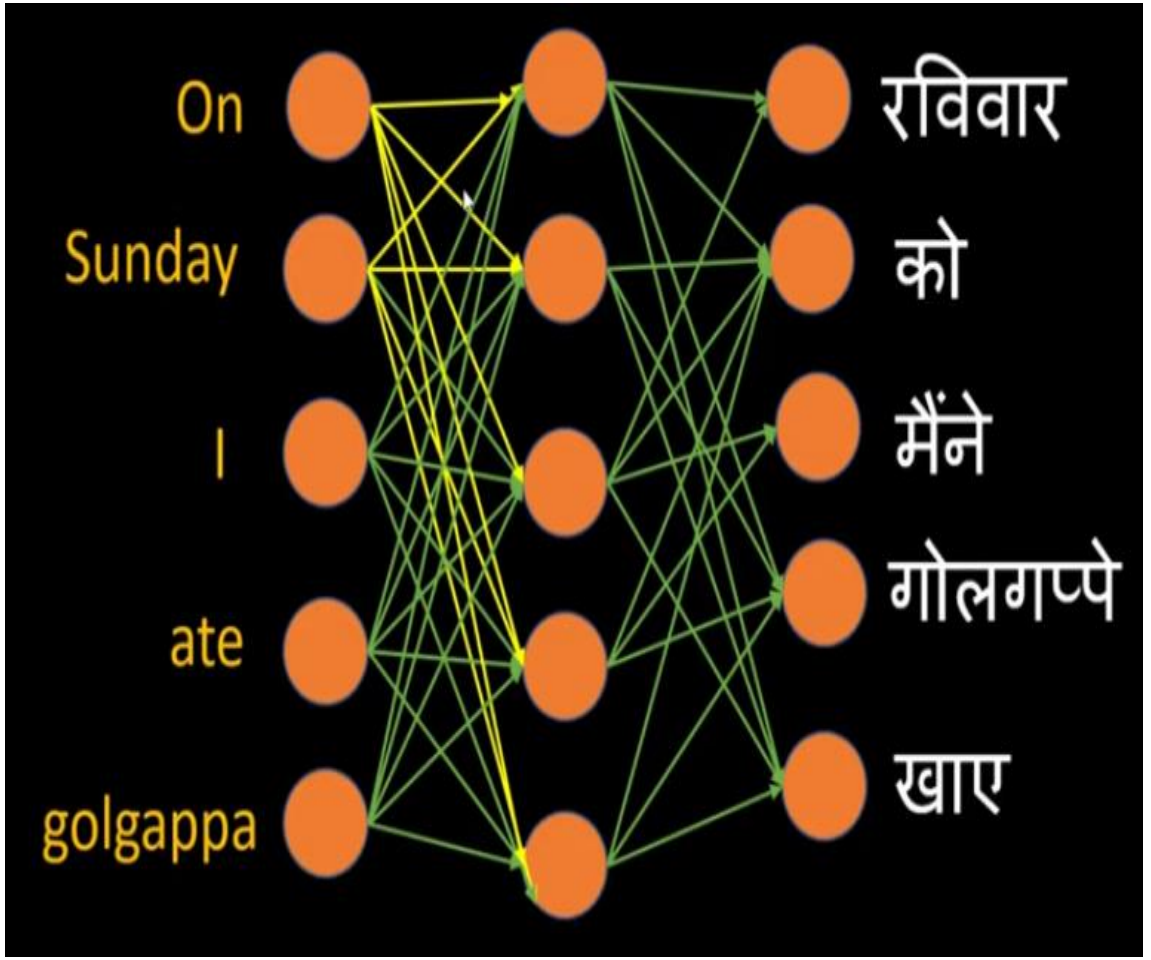
| 0 | 1 | 2 | Age | Salary |
|---|---|---|---|---|
| 1 | 0 | 0 | 44 | 72000 |
| 0 | 0 | 1 | 34 | 65000 |
| 0 | 1 | 0 | 46 | 98000 |
| 0 | 0 | 1 | 35 | 45000 |
| 0 | 1 | 0 | 23 | 34000 |


25000 words in vocabulary
how → [0,0,0,..,1,0,0,...,0 ]
are → [0, 1,0,0,0,..0,0,...,0 ]
you? → [0, 0,0,0,..0,0,1,0,0 ]


25000 words in vocabulary
how → [0,0,0,..,1,0,0,...,0 ]
are → [0, 1,0,0,0,..0,0,...,0 ]
you? → [0, 0,0,0,..0,0,1,0,0 ]
42000 words in vocabulary
[0,0,0,..,1,0,0,...,0 ]
[0,0,0,..,1,0,0,...,0 ]
[0,0,0,..,1,0,0,...,0 ]
क्या
हाल
है?

# No parameter sharing



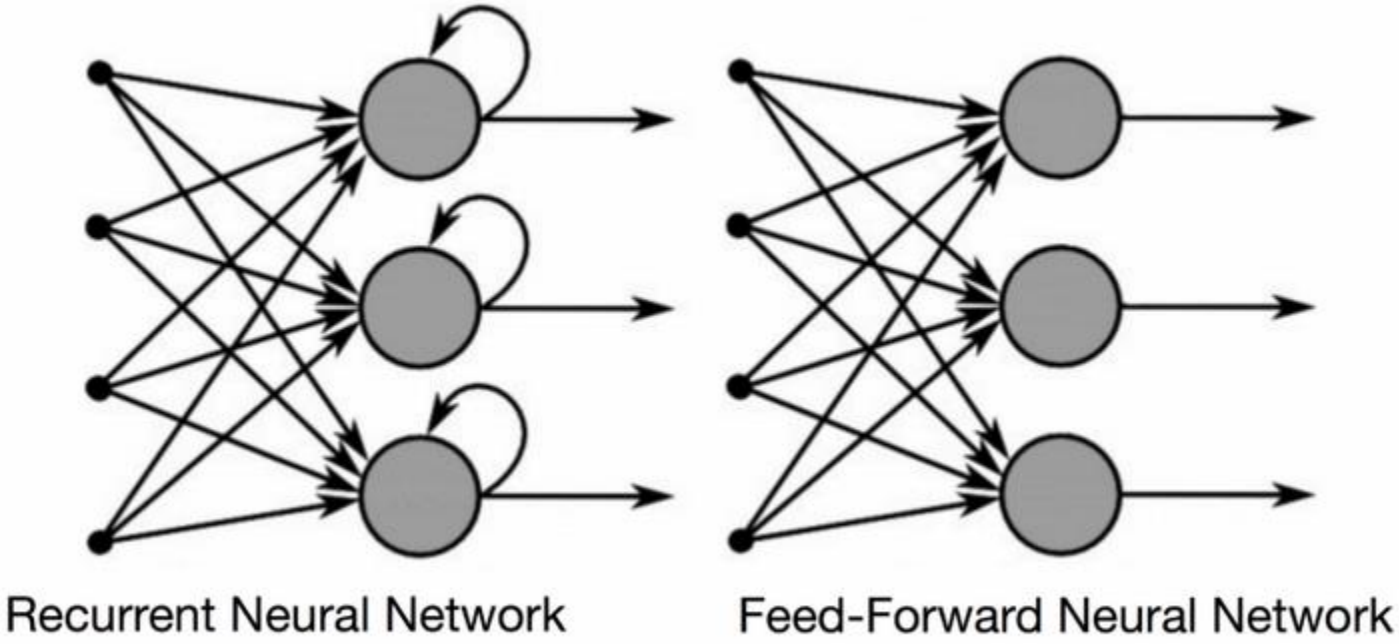



- Named Entity Recognition

*Mark Zuckerberg is one of the founders of Facebook, a company from the United States.*

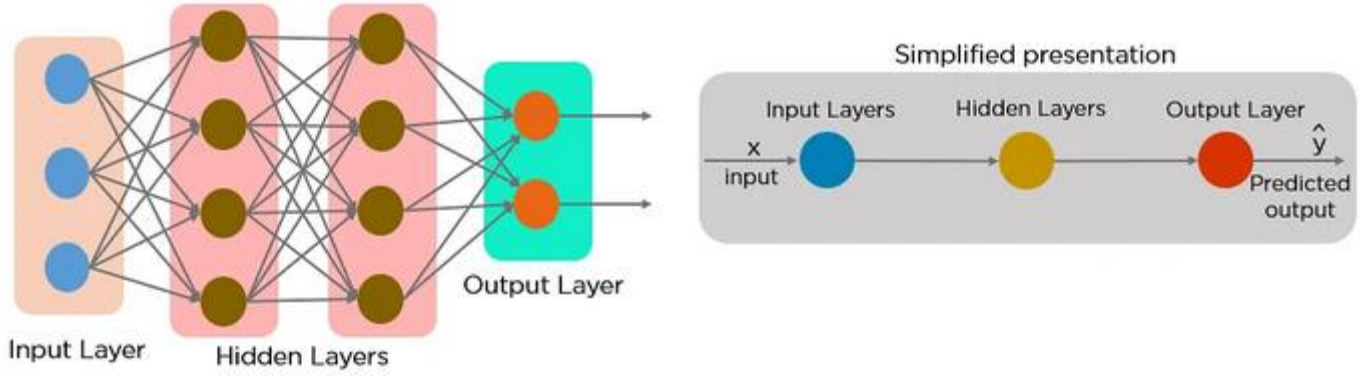
- *“Person”: Mark Zuckerberg*
- *“Company”: Facebook*
- *“Location”: United States*

The difference in information flow between a RNN and a feed-forward neural network.



# Feed-forward neural network

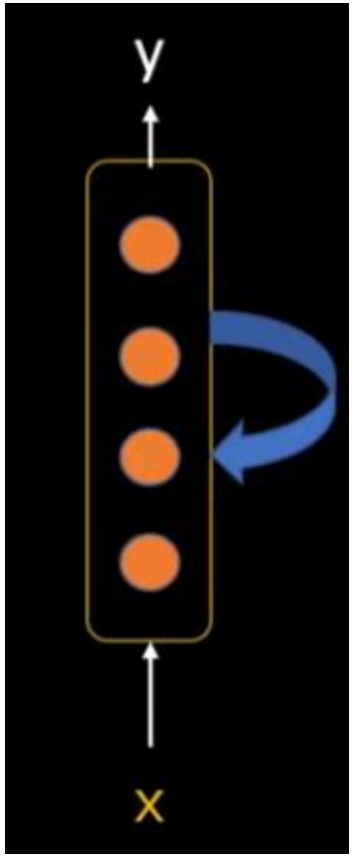
- A feed-forward neural network allows information to flow only in the forward direction, from the input nodes, through the hidden layers, and to the output nodes. There are no cycles or loops in the network.




Feed-forward Neural Network

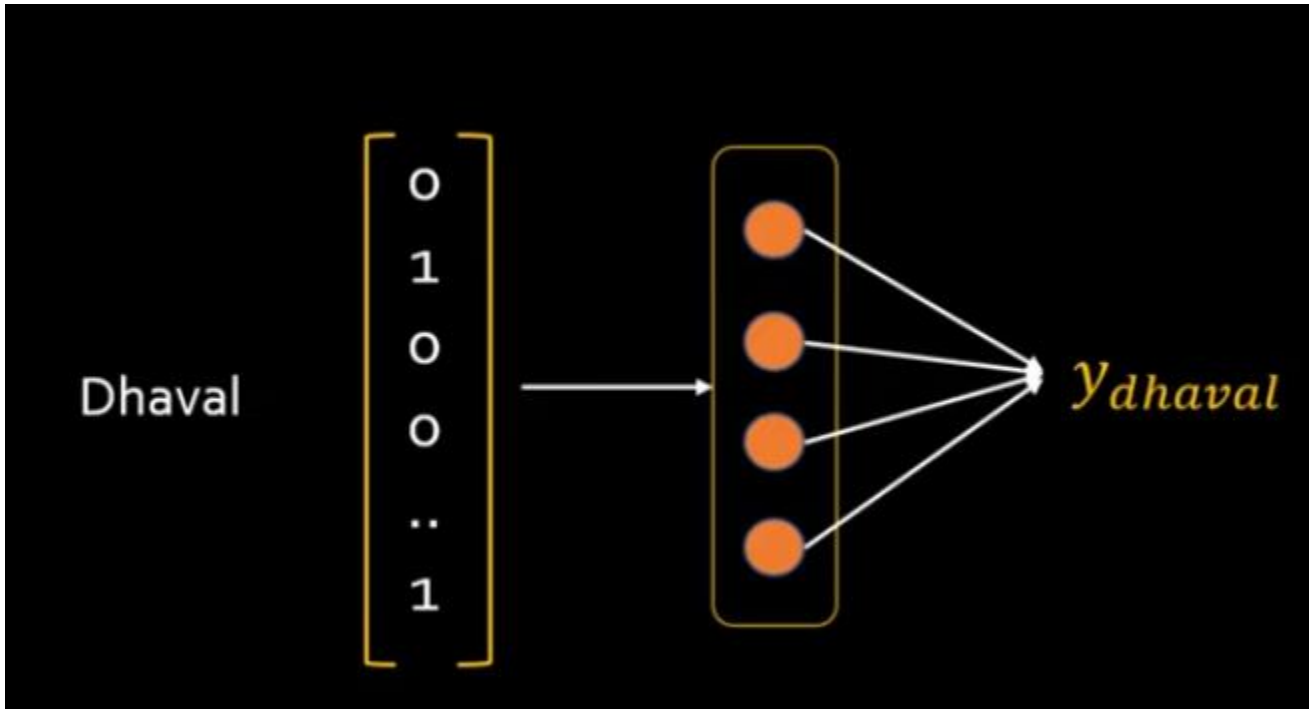
- In a feed-forward neural network, the decisions are based on the current input.
- It doesn’t memorize the past data, and there’s no future scope.
- Feed-forward neural networks are used in general regression and classification problems.

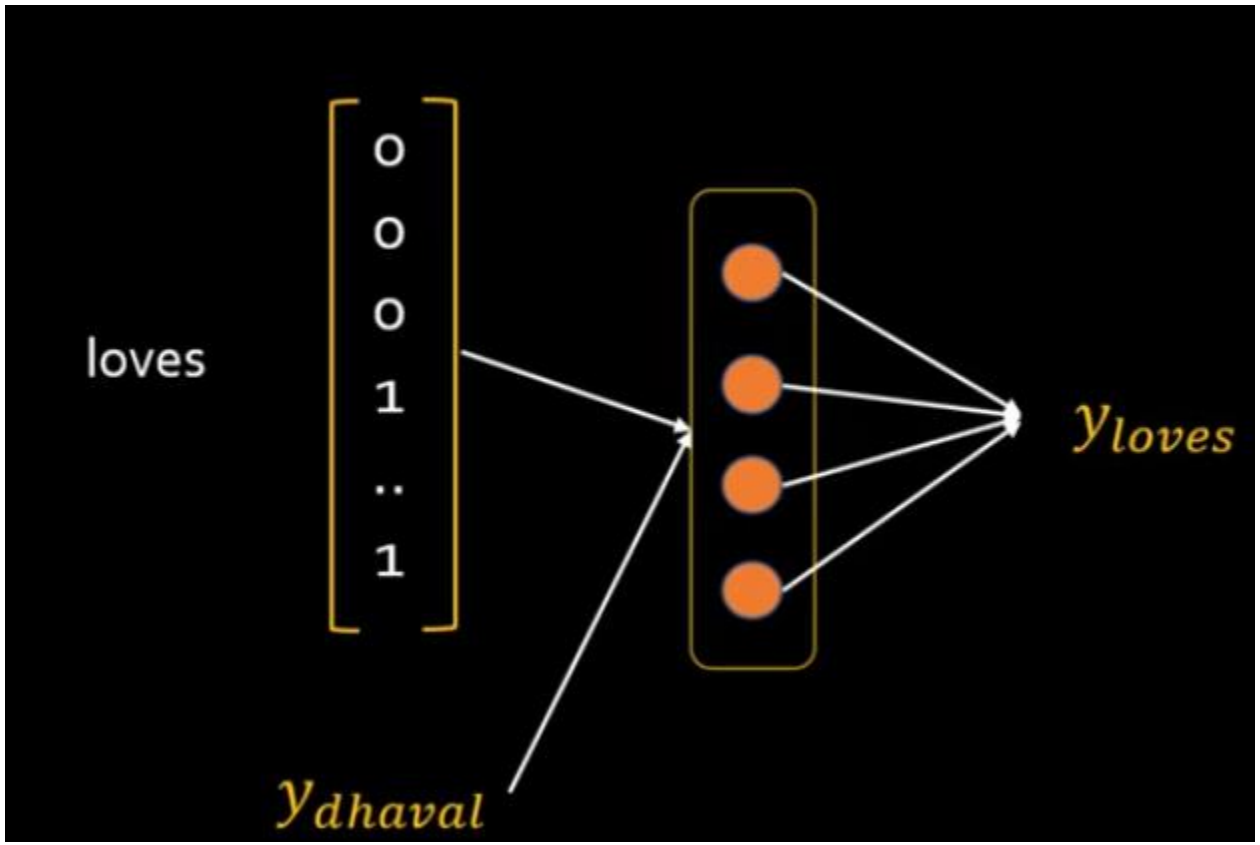
# Generic representation of RNN

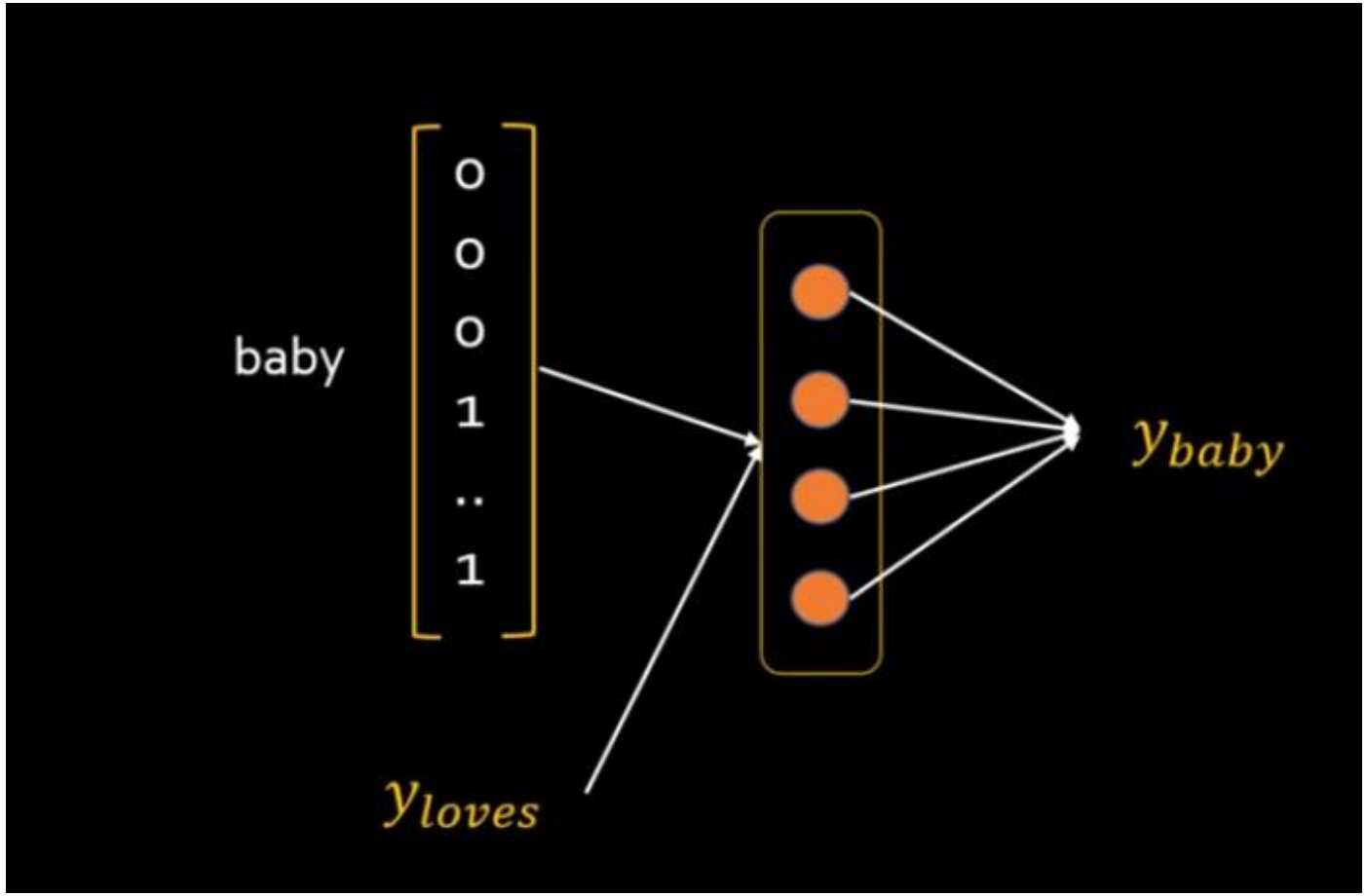


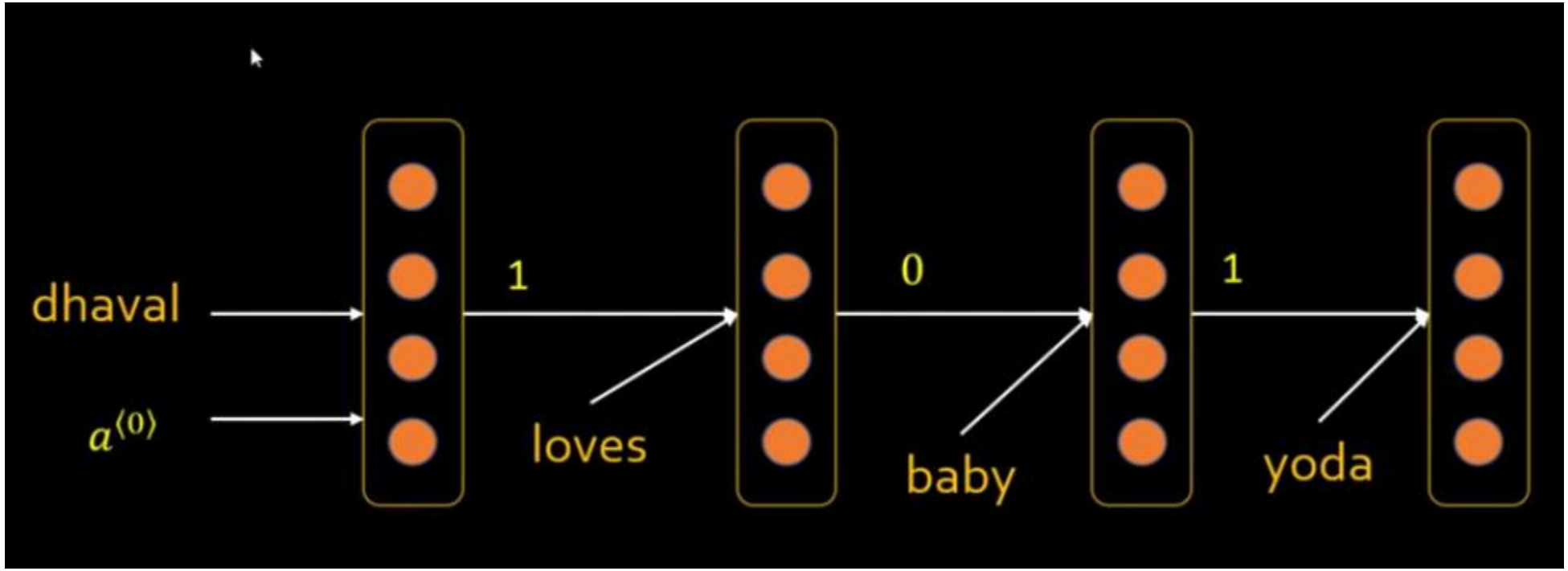

It produces output, copies that output and loops it back into the network.

# Dhaval loves baby yoda




loves
0
0
0
1
..
1
$y_{loves}$
$y_{dhaval}$


baby
0
0
0
1
..
1
$y_{baby}$
$y_{loves}$


dhaval
$a^{\langle 0 \rangle}$
1
loves
0
baby
1
yoda

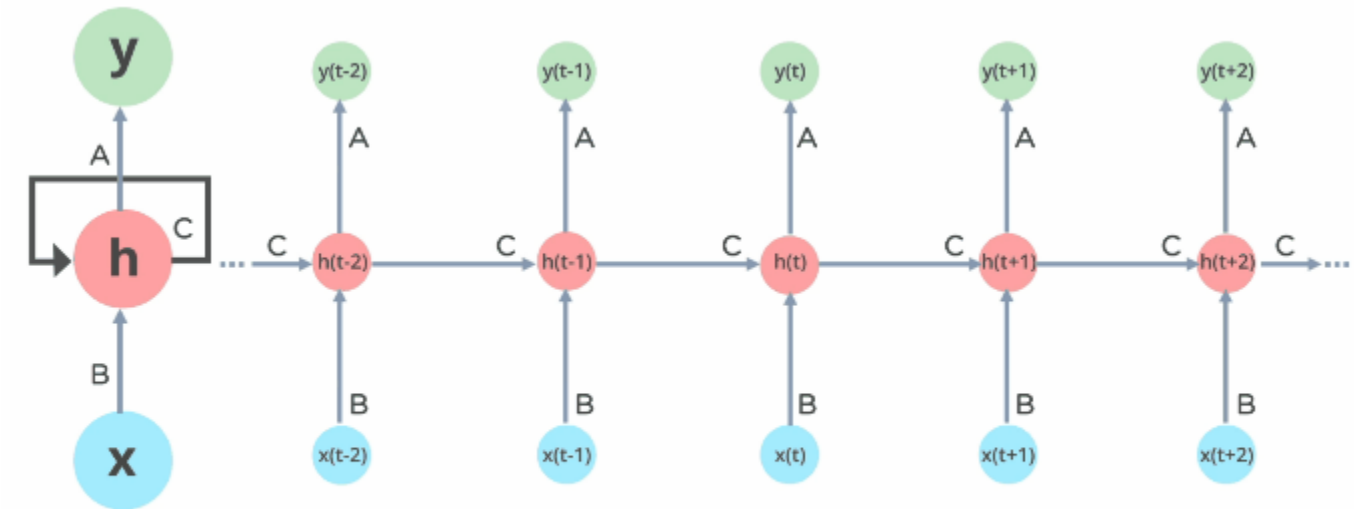
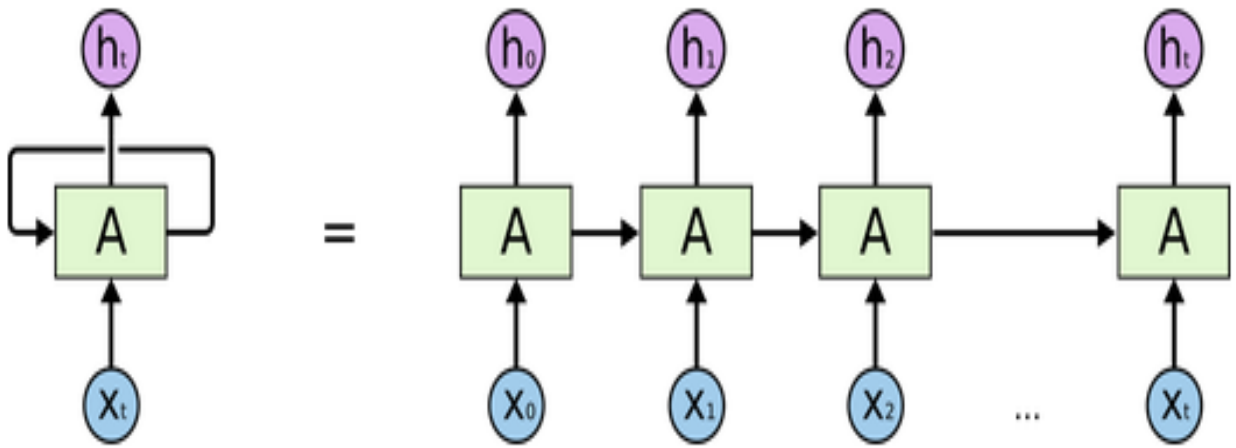



Fig: Working of Recurrent Neural Network

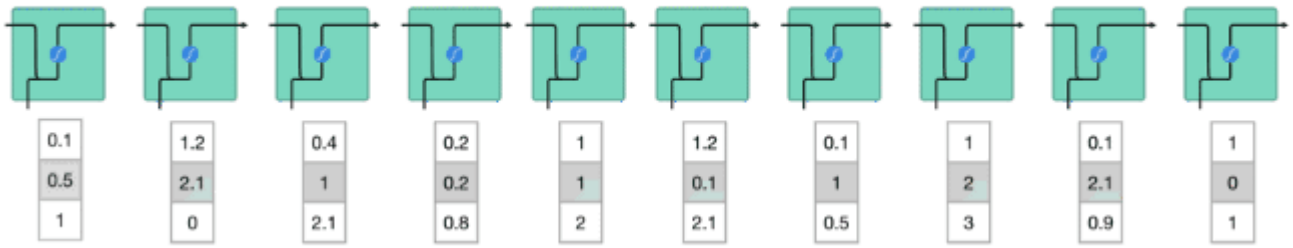



An unrolled recurrent neural network.

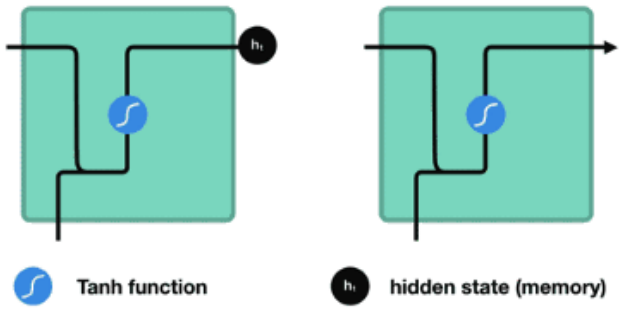
- A RNN has two inputs:
- the present and
- the recent past.
- In a RNN the information cycles through a loop. When it makes a decision, it considers the current input and also what it has learned from the inputs it received previously. So a RNN can do things which other algorithms can not.

# How RNN works

First words get transformed into machine-readable vectors. Then the RNN processes the sequence of vectors one by one.

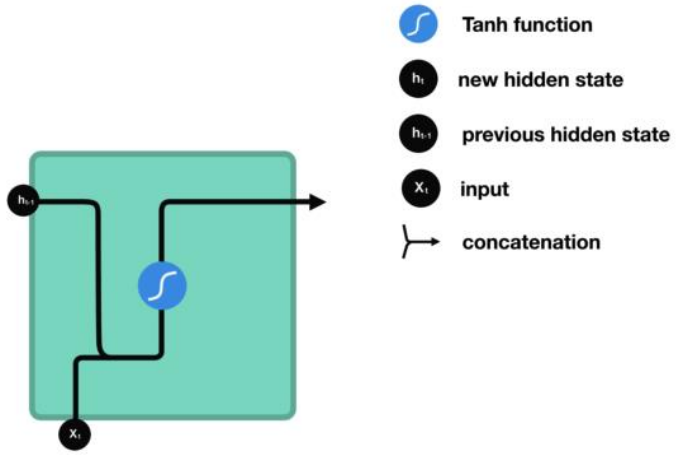


- While processing, it passes the previous hidden state to the next step of the sequence. The hidden state acts as the neural networks memory. It holds information on previous data the network has seen before.



# How RNN would calculate the hidden state?

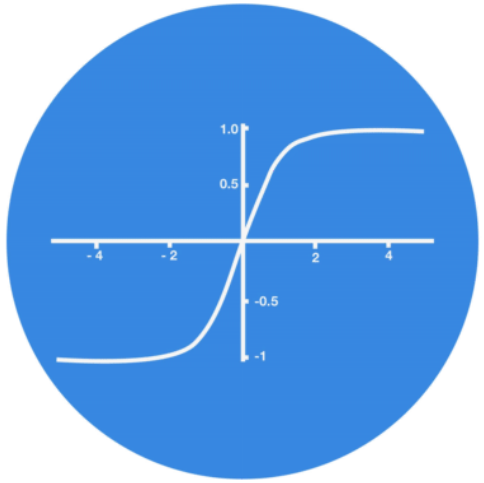
- First, the input and previous hidden state are combined to form a vector. That vector now has information on the current input and previous inputs. The vector goes through the tanh activation, and the output is the new hidden state, or the memory of the network.



- **Tanh activation**

- When vectors are flowing through a neural network, it undergoes many transformations due to various math operations. The tanh activation is used to help regulate the values flowing through the network. The tanh function squishes values to always be between -1 and 1.

| |
|---|
| 5 |
| 0.1 |
| -0.5 |



# How Does Recurrent Neural Networks Work?

- In Recurrent Neural networks, the information cycles through a loop to the middle hidden layer.
- The Recurrent Neural Network will standardize the different activation functions and weights and biases so that each hidden layer has the same parameters. Then, instead of creating multiple hidden layers, it will create **one hidden layer and loop over it** as many times as required.
- The input layer ‘x’ takes in the input to the neural network and processes it and passes it onto the middle layer.
- The middle layer or hidden layer, has its own activation function, weights and biases. Each recurrent layer has two sets of weights; one for the input and the second one for the hidden unit.
- The last feedforward layer, which computes the final output for the kth time step is just like an ordinary layer of a traditional feedforward network

- **The Activation Function**
- **Any of the following activation functions can be used in the recurrent neural network like**
- **Sigmoid function:** $\frac{1}{1+e^{-x}}$
- **Tanh function:** $\frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$
- **Relu function: max(0,x)**

# Applications of Recurrent Neural Networks

## Image Captioning

RNNs are used to caption an image by analyzing the activities present.



"A Dog catching a ball in mid air"

## Time Series Prediction

Any time series problem, like predicting the prices of stocks in a particular month, can be solved using an RNN.

## Natural Language Processing

Text mining and Sentiment analysis can be carried out using an RNN for Natural Language Processing (NLP).




When it rains, look for rainbows.
When it's dark, look for stars.

Positive Sentiment

## Machine Translation

Given an input in one language, RNNs can be used to translate the input into different languages as output.



Here the person is speaking in English and it is getting translated into Chinese, Italian, French, German and Spanish languages

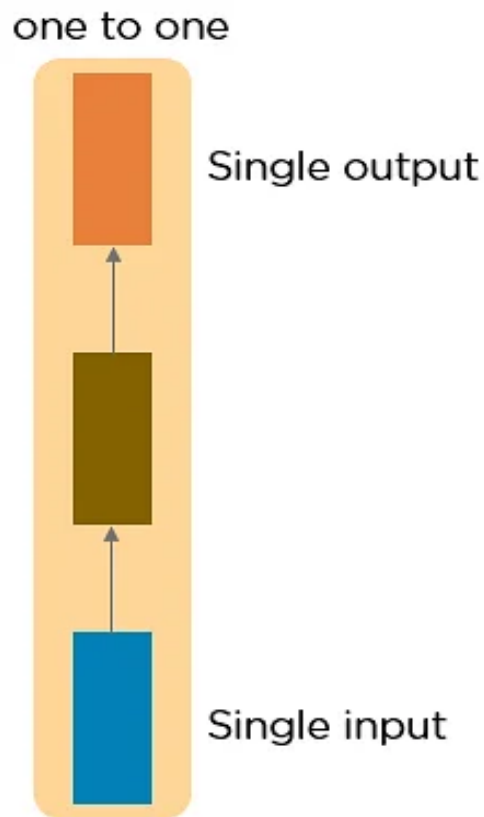
# Types of RNNs

- One to One
- One to Many
- Many to One
- Many to Many

- While feed-forward neural networks map one input to one output, RNNs can map one to many, many to many (translation) and many to one (classifying a voice).

## One to One RNN

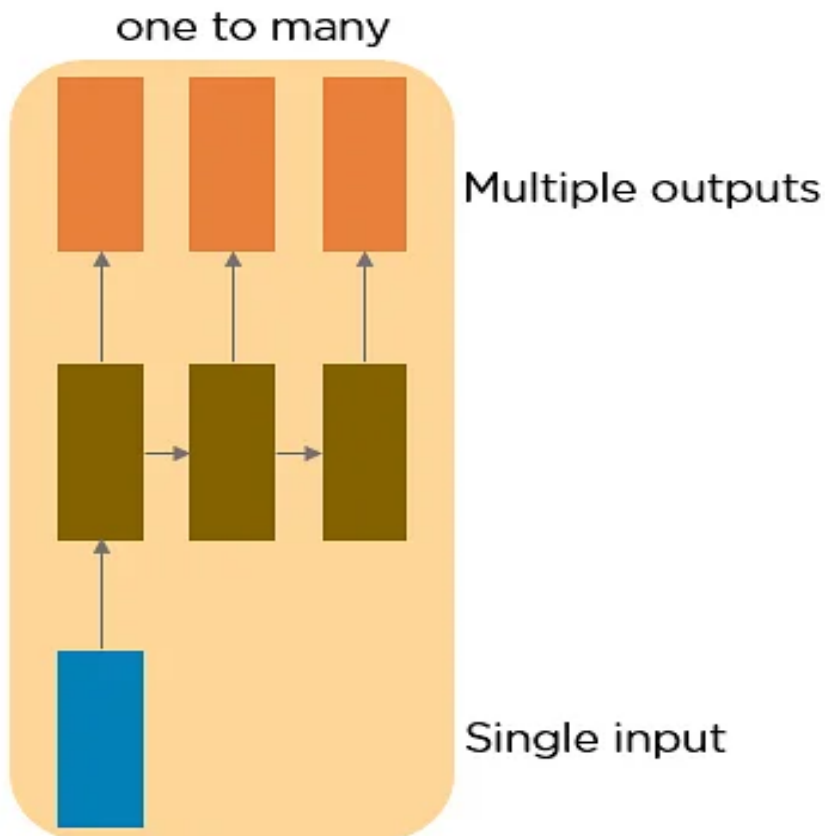
This type of neural network is known as the Vanilla Neural Network. It's used for general machine learning problems, which has a single input and a single output.




Activate
Go to Settin

## One to Many RNN

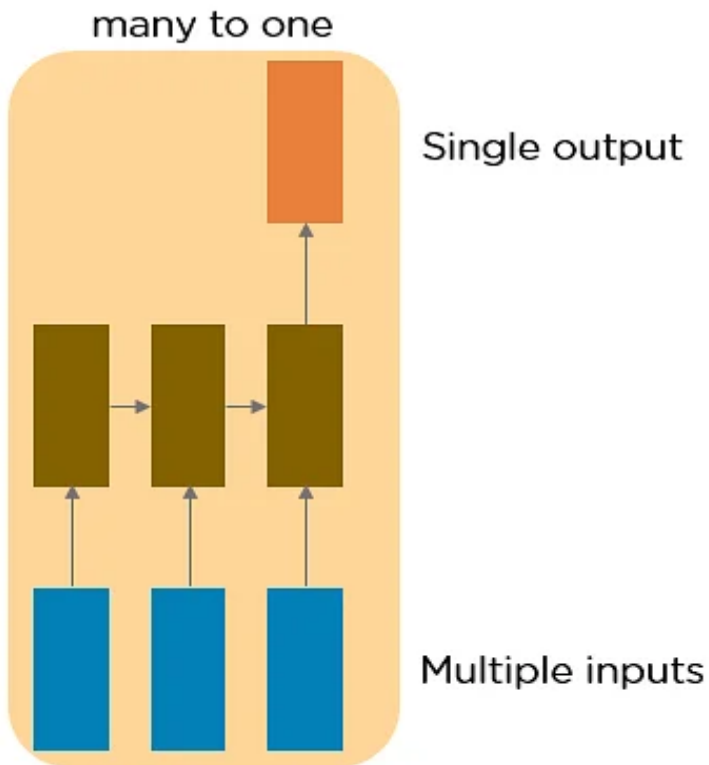
This type of neural network has a single input and multiple outputs. An example of this is the image caption.




Activate W
Go to Settings

## Many to One RNN

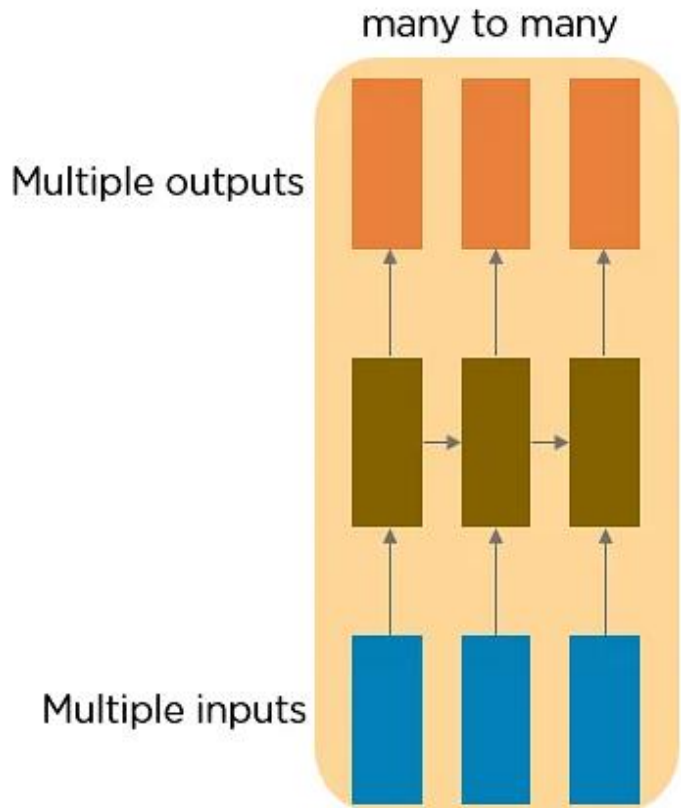
This RNN takes a sequence of inputs and generates a single output. Sentiment analysis is a good example of this kind of network where a given sentence can be classified as expressing positive or negative sentiments.




Activate Win
Go to Settings to

## Many to Many RNN

This RNN takes a sequence of inputs and generates a sequence of outputs. Machine translation is one of the examples.




Activate Win
Go to Settings to

# Problems associated with Recurrent Neural Networks(RNN):

- The recurrent network is a simple and powerful model.In practice, it is hard to train RNN properly. The main reasons for this is
- Vanishing gradient problem and
- Short term memory problem.

# 1. RNN suffer from Vanishing Gradient Problem

- The gradient descent algorithm finds the global minimum of the cost function of the network. Gradients are values used to update a neural networks weights.
- new weight = old weight – learning rate * gradient

- Backpropagation is used for calculating the gradient of an error function with respect to a neural network’s weights.
- The algorithm works its way backwards through the various layers of gradients to find the partial derivative of the errors with respect to the weights.  These weights are used to decrease error margins when training.

- During back propagation, recurrent neural networks suffer from the vanishing gradient problem.
- The vanishing gradient problem is when the gradient shrinks as it back propagates through time. If a gradient value becomes extremely small, it doesn’t contribute too much learning.

# 2.RNN has Short-term Memory Problem

- Recurrent Neural Networks suffer from short-term memory. If a sequence is long enough, they'll have a hard time carrying information from earlier time steps to later ones.
- RNNs may forget what it has seen in longer sequences, as it has a short-term memory.

- To predict the last word in "the clouds are in the ……..
- To predict the last word in the text "I grew up in France… I speak fluent ……

- A feed-forward neural network assigns a weight matrix to its inputs and then produces the output.
- RNNs apply weights to the current and also to the previous input.

**Note:**

- A gradient is a partial derivative with respect to its inputs.
- A gradient measures how much the output of a function changes if you change the inputs a little bit.
- A gradient can also be thought of as the slope of a function. The higher the gradient, the steeper the slope and the faster a model can learn. But if the slope is zero, the model stops learning.

# Back Propagation through time

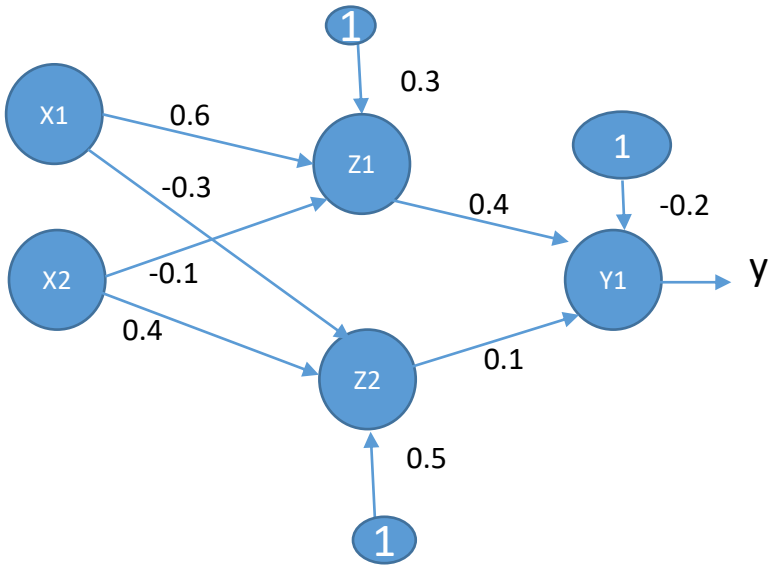
- Recurrent Neural Networks are those networks that deal with sequential data.
- They predict outputs using not only the current inputs but also by taking into consideration those that occurred before it. In other words, the current output depends on current input as well as a memory element (which takes into account the past inputs).
- For training such networks, we use backpropagation with some modification which is called Back Propagation Through Time or BPTT algorithm.

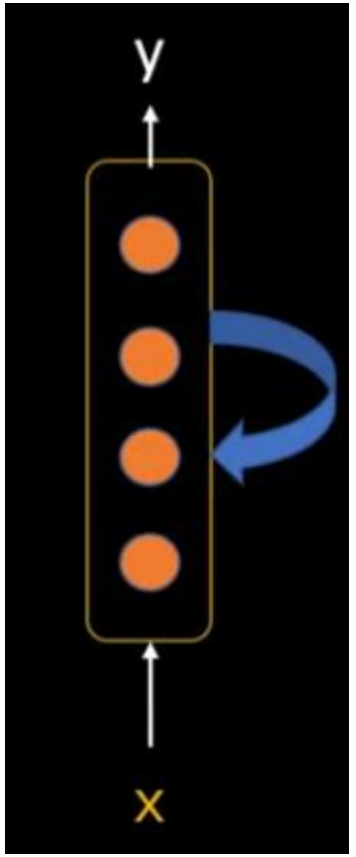
# BackPropagation

- It is a  training algorithm for updating network weights to minimize error.
- The goal of the backpropagation training algorithm is to modify the weights of a neural network in order to minimize the error of the network outputs compared to some expected output in response to corresponding inputs.
- It is a supervised learning algorithm that allows the network to be corrected with regard to the specific errors made.

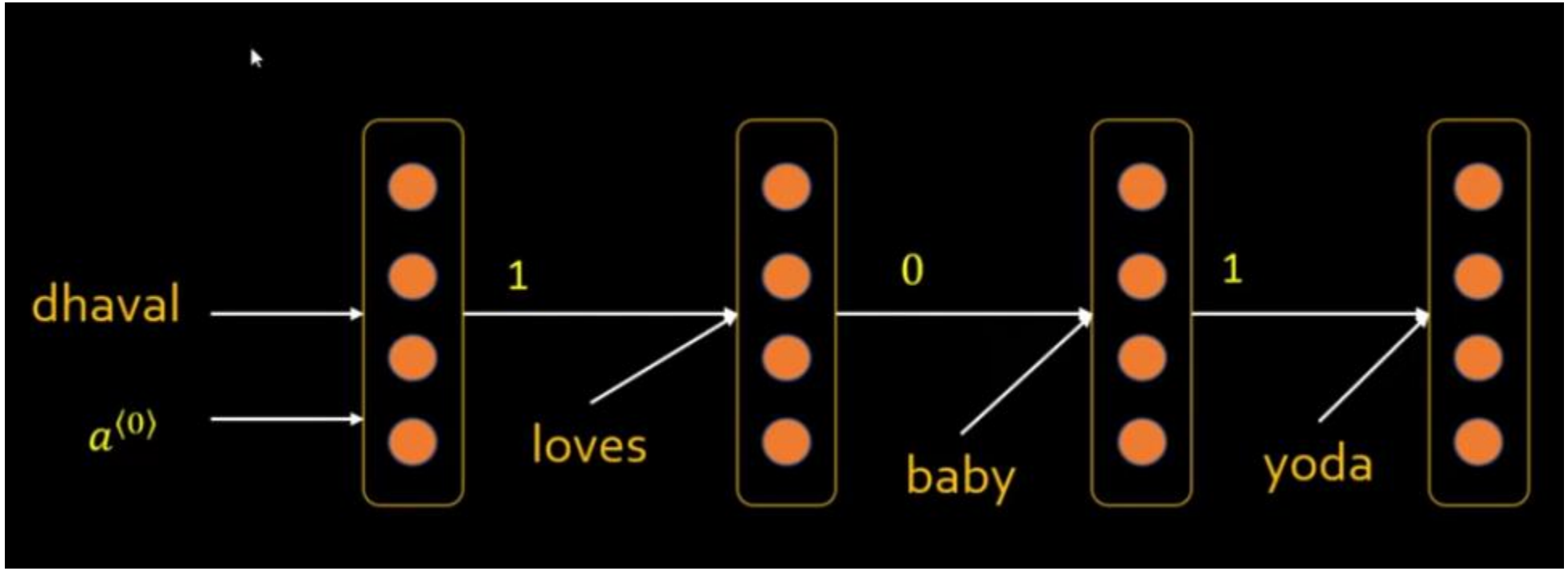
- The general Backpropagation algorithm is as follows:

1. Present a training input pattern and propagate it through the network to get an output.
2. Compare the predicted outputs to the expected outputs and calculate the error.
3. Calculate the derivatives of the error with respect to the network weights.
4. Adjust the weights to minimize the error.
5. Repeat.

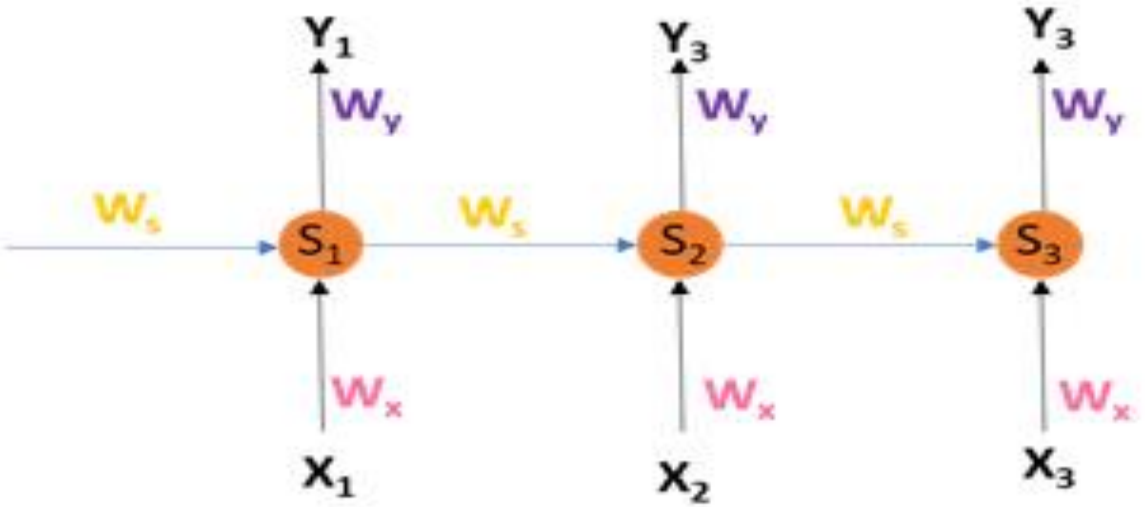

1
0.3
X1
0.6
Z1
1
-0.3
0.4
-0.2
X2
-0.1
Y1
y
0.4
Z2
0.1
0.5
1


y
x

Backpropagation Through Time, or BPTT, is the training algorithm used to update weights in recurrent neural networks.




A recurrent neural network is shown one input each timestep and predicts one output.




*RNN Architecture*

*S1*, *S2*, *S3* are the hidden states or memory units at time *t1*, *t2*, *t3* respectively, and *Ws* is the weight matrix associated with it.

*X1*, *X2*, *X3* are the inputs at time *t1*, *t2*, *t3* respectively, and *Wx* is the weight matrix associated with it.

*Y1*, *Y2*, *Y*3 are the outputs at time *t1*, *t2*, *t3* respectively, and *Wy* is the weight matrix associated with it.

- For any time, t, we have the following two equations:

- $$S_t = g_1(W_x x_t + W_s S_{t-1})$$
$$Y_t = g_2(W_Y S_t)$$

where g1 and g2 are activation functions.

Let the error function be:

$$E_t = (d_t - Y_t)^2$$

, so at t =3,

$$E_3 = (d_3 - Y_3)^2$$

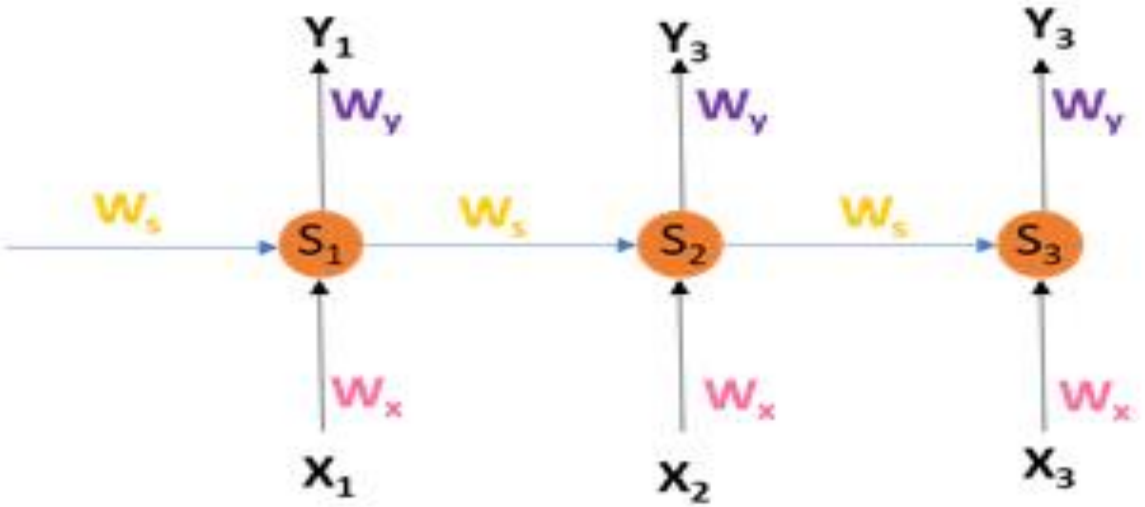
We are using the squared error here, where *d3* is the desired output at time *t = 3*.

To perform back propagation, we have to adjust the weights associated with inputs( $W_x$), the memory units ($W_s$) and the outputs($W_y$).

# BPTT

1. Present a sequence of timesteps of input and output pairs to the network.
2. Unroll the network then calculate and accumulate errors across each timestep.
3. Roll-up the network and update weights.
4. Repeat.

- BPTT can be computationally expensive as the number of timesteps increases.




*RNN Architecture*

*S1*, *S2*, *S3* are the hidden states or memory units at time *t1*, *t2*, *t3* respectively, and *Ws* is the weight matrix associated with it.

*X1*, *X2*, *X3* are the inputs at time *t1*, *t2*, *t3* respectively, and *Wx* is the weight matrix associated with it.

*Y1*, *Y2*, *Y*3 are the outputs at time *t1*, *t2*, *t3* respectively, and *Wy* is the weight matrix associated with it.

# To Adjust Wy




**Formula:**

$$\frac{\partial E_3}{\partial W_y} = \frac{\partial E_3}{\partial Y_3} \cdot \frac{\partial Y_3}{\partial W_Y}$$

*Error E3* is a function of calculated output *Y3*. Hence, we differentiate *E3* w.r.t *Y3*.
*Y3* is a function of $W_Y$. Hence, we differentiate *Y3* w.r.t $W_Y$.

# To Adjust Ws

$$\frac{\partial E_3}{\partial W_S} = \left(\frac{\partial E_3}{\partial Y_3} \cdot \frac{\partial Y_3}{\partial S_3} \cdot \frac{\partial S_3}{\partial W_S}\right) +$$

$$\left(\frac{\partial E_3}{\partial Y_3} \cdot \frac{\partial Y_3}{\partial S_3} \cdot \frac{\partial S_3}{\partial S_2} \cdot \frac{\partial S_2}{\partial W_S}\right) +$$

$$\left(\frac{\partial E_3}{\partial Y_3} \cdot \frac{\partial Y_3}{\partial S_3} \cdot \frac{\partial S_3}{\partial S_2} \cdot \frac{\partial S_2}{\partial S_1} \cdot \frac{\partial S_1}{\partial W_S}\right)$$

$E_3$ is a function of $Y_3$. Hence, differentiate $E_3$ w.r.t $Y_3$.
$Y_3$ is a function of $S_3$. Hence, we differentiate $Y_3$ w.r.t $S_3$.
$S_3$ is a function of $W_S$. Hence, we differentiate $S_3$ w.r.t $W_S$.
we have to take the previous time steps also into consideration,. So, we differentiate (partially) the Error function with respect to memory units $S_2$ as well as $S_1$ taking into consideration the weight matrix $W_S$.
We have to keep in mind that a memory unit, say $S_t$ is a function of its previous memory unit $S_{t-1}$.

# General formula

$$\frac{\partial E_N}{\partial W_S} = \sum_{i=1}^{N} \frac{\partial E_N}{\partial Y_N} \cdot \frac{\partial Y_N}{\partial S_i} \cdot \frac{\partial S_i}{\partial W_S}$$

To Adjust $W_X$:

$$\frac{\partial E_3}{\partial W_X} = (\frac{\partial E_3}{\partial Y_3} \cdot \frac{\partial Y_3}{\partial S_3} \cdot \frac{\partial S_3}{\partial W_X}) +$$

$$(\frac{\partial E_3}{\partial Y_3} \cdot \frac{\partial Y_3}{\partial S_3} \cdot \frac{\partial S_3}{\partial S_2} \cdot \frac{\partial S_2}{\partial W_X}) +$$

$$(\frac{\partial E_3}{\partial Y_3} \cdot \frac{\partial Y_3}{\partial S_3} \cdot \frac{\partial S_3}{\partial S_2} \cdot \frac{\partial S_2}{\partial S_1} \cdot \frac{\partial S_1}{\partial W_X})$$

- This method of Back Propagation through time (BPTT) can be used up to a limited number of time steps like 8 or 10.
- If we back propagate further, the gradient becomes too small. This problem is called the “Vanishing gradient” problem.

-One famous solutions to this problem is by using Long Short-Term Memory (LSTM for short) cells instead of the traditional RNN cells. Then another another problem called the exploding gradient problem may occur, where the gradient grows uncontrollably large. A method called gradient clipping can be used where in each time step, we can check if the gradient > threshold. If yes, then normalize it.

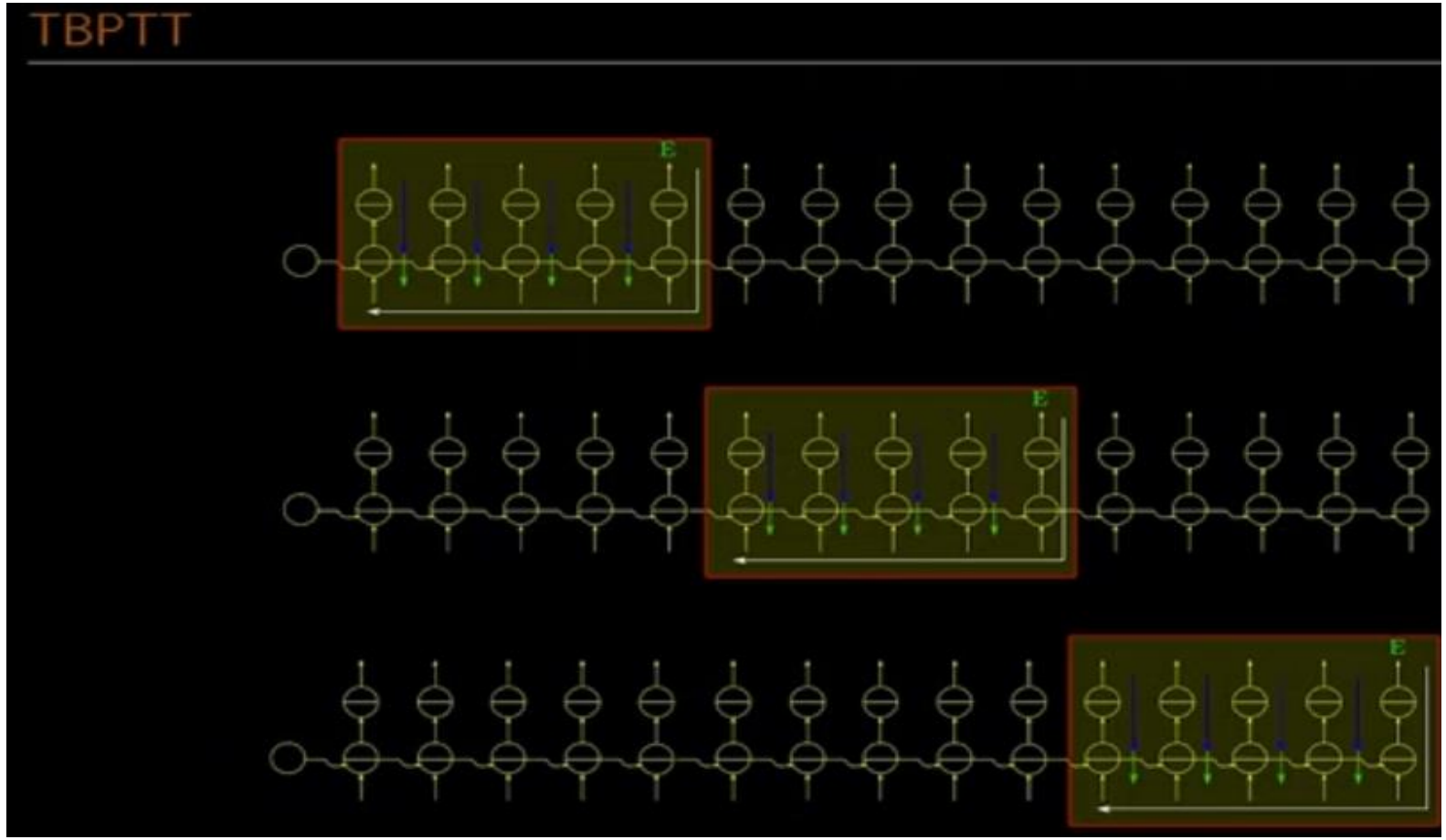
# Truncated Backpropagation Through Time

- Truncated Backpropagation Through Time, or TBPTT, is a modified version of the BPTT training algorithm for recurrent neural networks where the sequence is processed one timestep at a time and periodically (k1 timesteps) the BPTT update is performed back for a fixed number of timesteps (k2 timesteps).

# Truncated Backpropagation Through Time Algorithm:

1. Present a sequence of k1 timesteps of input and output pairs to the network.
2. Unroll the network then calculate and accumulate errors across k2 timesteps.
3. Roll-up the network and update weights.
4. Repeat

- The TBPTT algorithm requires the consideration of two parameters:
- **k1**: The number of forward-pass timesteps between updates. Generally, this influences how slow or fast training will be, given how often weight updates are performed.
- **k2**: The number of timesteps to which to apply BPTT.


TBPTT

# Advantages of RNN

- RNNs have various advantages such as:
  - Ability to handle sequence data.
  - Ability to handle inputs of varying lengths.
  - Ability to store or ‘memorize’ historical information.

# Drawbacks of RNN

- The computation can be very slow.
- RNN can not store information for a longer period of time. At times, a reference to certain information stored quite a long time ago is required to predict the current output. But RNNs are absolutely incapable of handling such “long-term dependencies”.
- RNN has no  control over which part of the context needs to be carried forward and how much of the past needs to be ‘forgotten’.
- Exploding and vanishing gradients problems which occur during the training process of a network through backtracking.

# Exploding and Vanishing Gradients:

- During the training process of a network, the main goal is to minimize loss (in terms of error or cost) observed in the output when training data is sent through it.
- We calculate the gradient, that is, loss with respect to a particular set of weights, adjust the weights accordingly and repeat this process until we get an optimal set of weights for which loss is minimum. This is the concept of backpropagation.
- Sometimes, it so happens that the gradient is almost negligible. The gradient of a layer depends on certain components in the successive layers. If some of these components are small (less than 1), the result obtained, which is the gradient, will be even smaller. This is known as the scaling effect. When this gradient is multiplied with the learning rate which is in itself a small value, it results in a smaller value. As a consequence, the updations in weights is quite small, producing almost the same output as before which is known as vanishing Gradients. Similarly, if the gradients are quite large in value due to the large values of components, the weights get updated to a value beyond the optimal value. This is known as the problem of exploding gradients.

# Reason for Short memory Problem of RNN

- During back propagation, recurrent neural networks suffer from the vanishing gradient problem. Gradients are values used to update a neural networks weights. The vanishing gradient problem is when the gradient shrinks as it back propagates through time. If a gradient value becomes extremely small, it doesn't contribute too much learning.




Gradient Update Rule

- So in recurrent neural networks, layers, usually the earlier layers, that get a small gradient update stops learning.  So because these layers don’t learn, RNN’s can forget what it seen in longer sequences, thus having a short-term memory.

# Why LSTM is required?

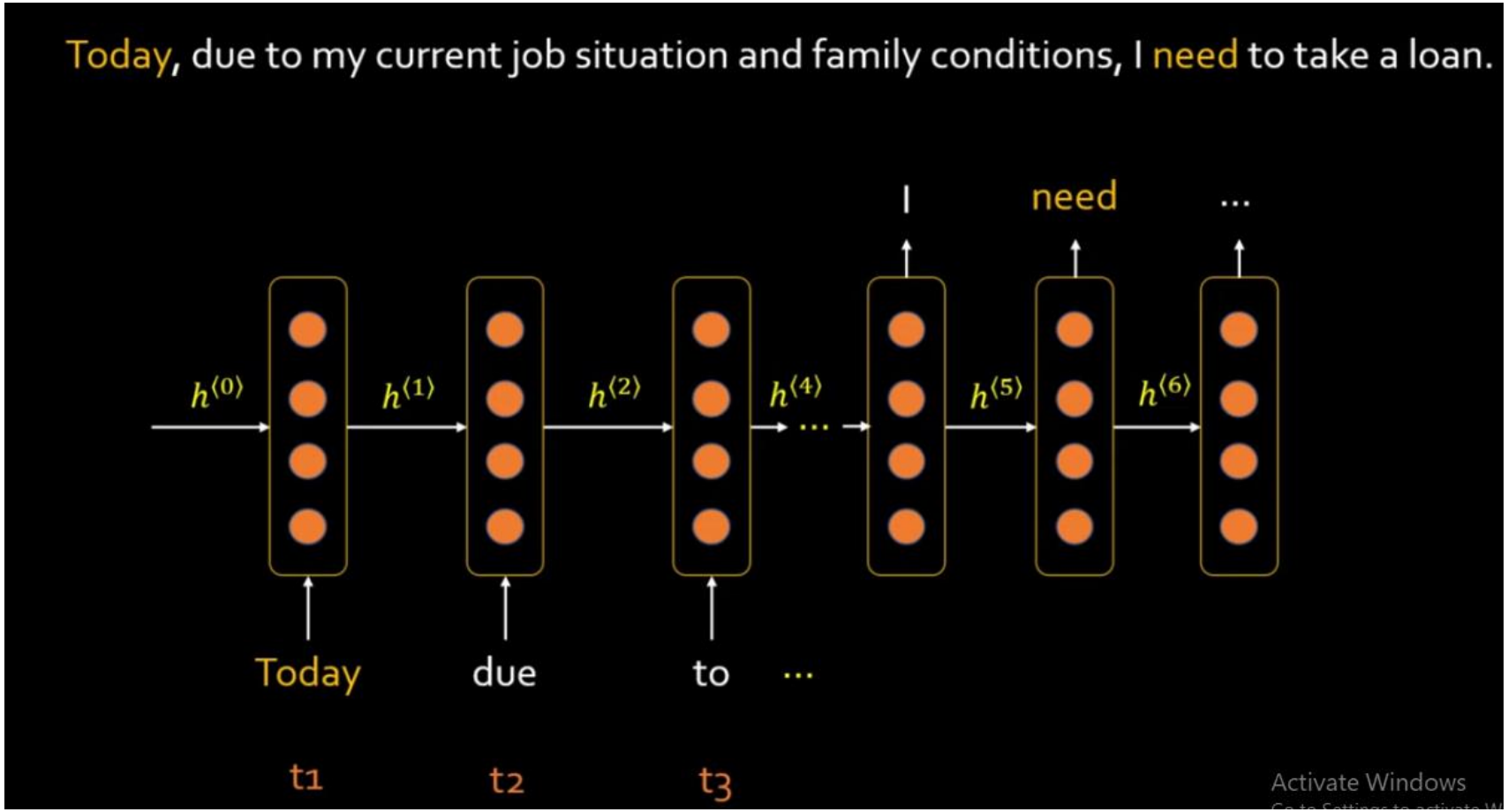
- Recurrent Neural Networks suffer from short-term memory. If a sequence is long enough, they'll have a hard time carrying information from earlier time steps to later ones. So if a paragraph of text is reqired to do predictions, RNN's may leave out important information from the beginning.
- LSTM 's and GRU's were created as the solution to short-term memory problem.
- Long Short Term Memory(LSTMs) are a special kind of RNN, capable of learning long-term dependencies.

Today, due to my current job situation and family conditions, I ...

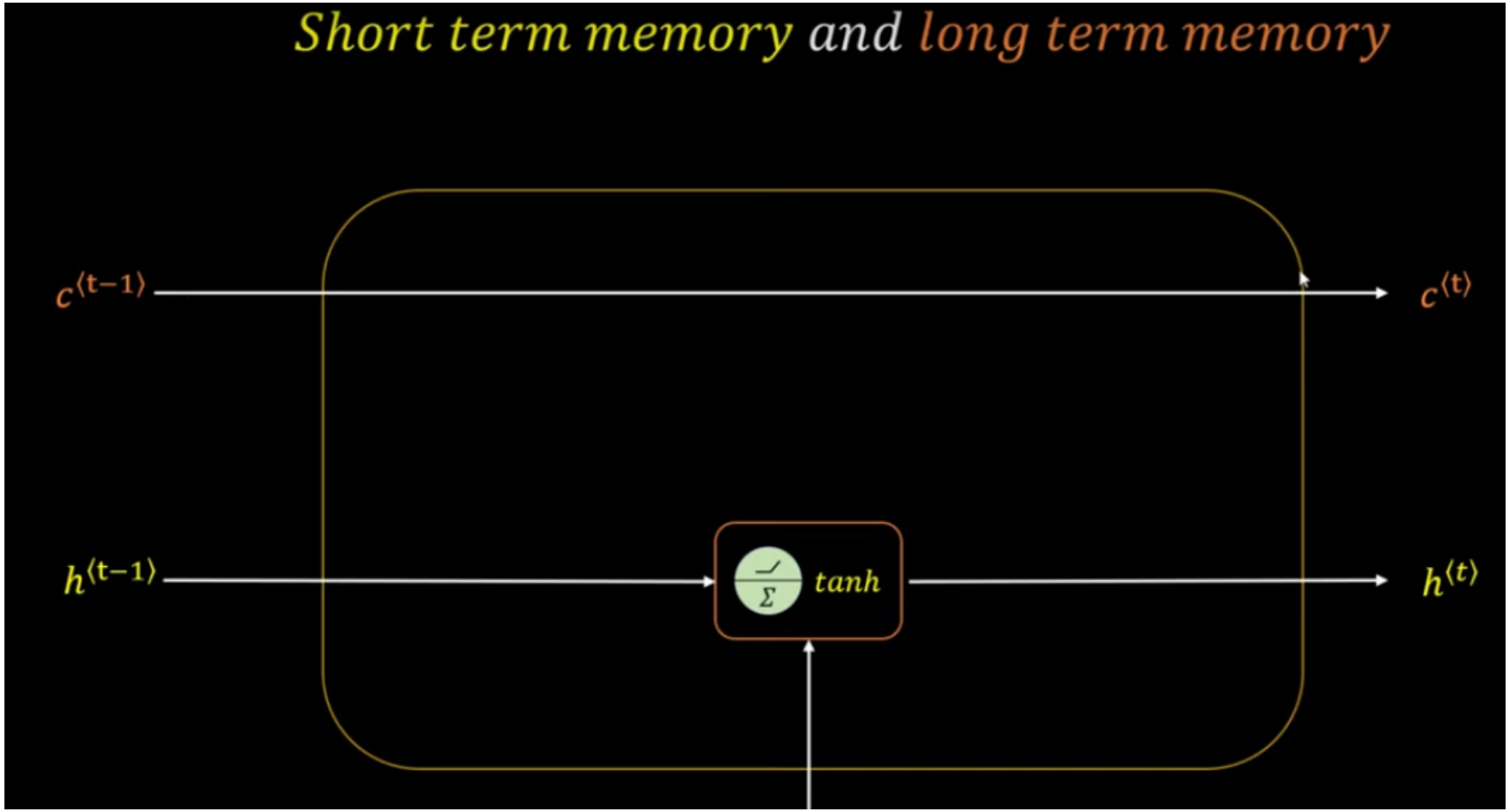
Last year, due to my current job situation and family conditions, I ..

Today, due to my current job situation and family conditions, I need to take a loan.

Last year, due to my current job situation and family conditions, I had to take a loan.

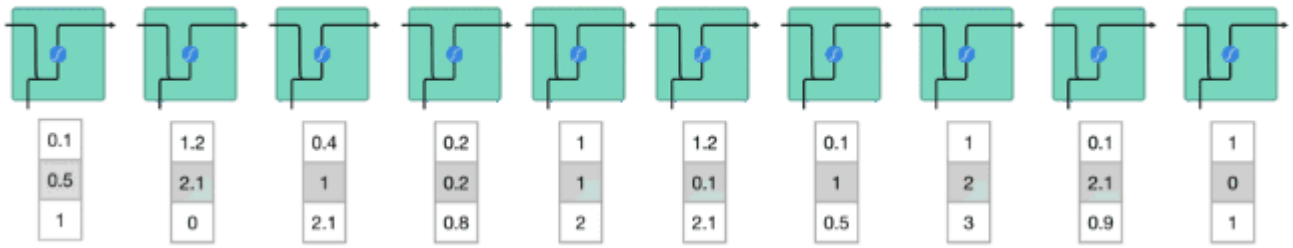



Activate Windows

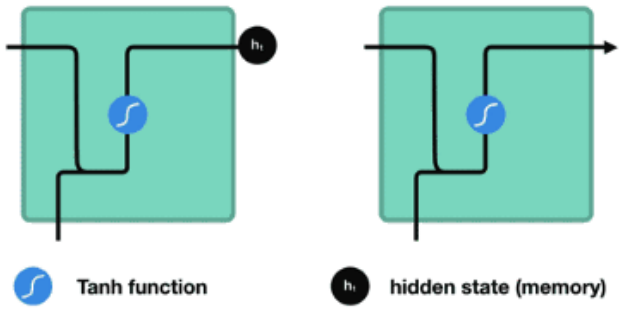

Short term memory and long term memory
$c^{\langle t-1 \rangle}$
$c^{\langle t \rangle}$
$h^{\langle t-1 \rangle}$
$\sigma$
$\Sigma$
tanh
$h^{\langle t \rangle}$

# How RNN works

First words get transformed into machine-readable vectors. Then the RNN processes the sequence of vectors one by one.

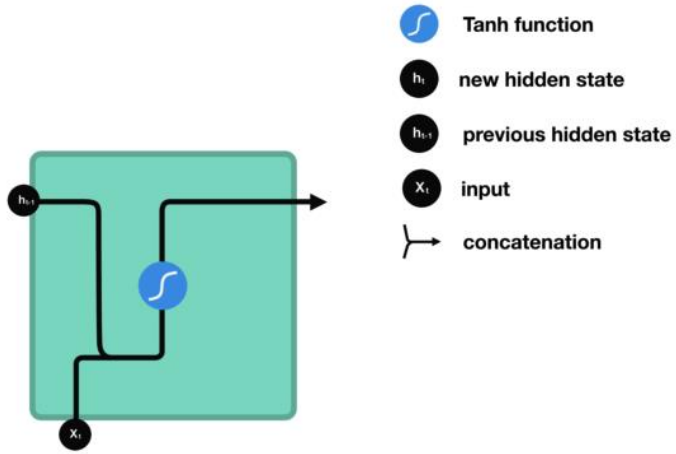


- While processing, it passes the previous hidden state to the next step of the sequence. The hidden state acts as the neural networks memory. It holds information on previous data the network has seen before.



# How RNN would calculate the hidden state?

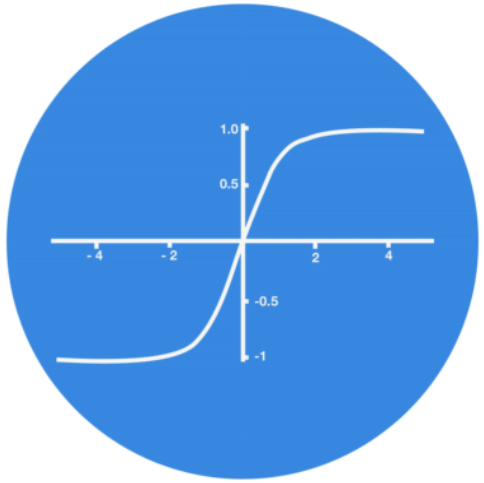
- First, the input and previous hidden state are combined to form a vector. That vector now has information on the current input and previous inputs. The vector goes through the tanh activation, and the output is the new hidden state, or the memory of the network.



- **Tanh activation**

- When vectors are flowing through a neural network, it undergoes many transformations due to various math operations. The tanh activation is used to help regulate the values flowing through the network. The tanh function squishes values to always be between -1 and 1.

| |
|---|
| 5 |
| 0.1 |
| -0.5 |



# Long Short-Term Memory(LSTM)

- Long Short-Term Memory is an advanced version of recurrent neural network (RNN) architecture and was mainly introduced to handle situations where RNNs fail.
- RNN is a network that works on the present input by taking into consideration the previous output (feedback) and storing in its memory for a short period of time (short-term memory).
- LSTMs enable RNNs to remember inputs over a long period of time. This is because LSTMs contain information in a memory, much like the memory of a computer. The LSTM can read, write and delete information from its memory. LSTM has internal mechanisms called gates that can regulate the flow of information. These gates can learn which data in a sequence is important to keep or throw away. By doing that, it can pass relevant information down the long chain of sequences to make predictions. It keeps only relevant information to make predictions, and forget non relevant data.

# LSTM Architecture

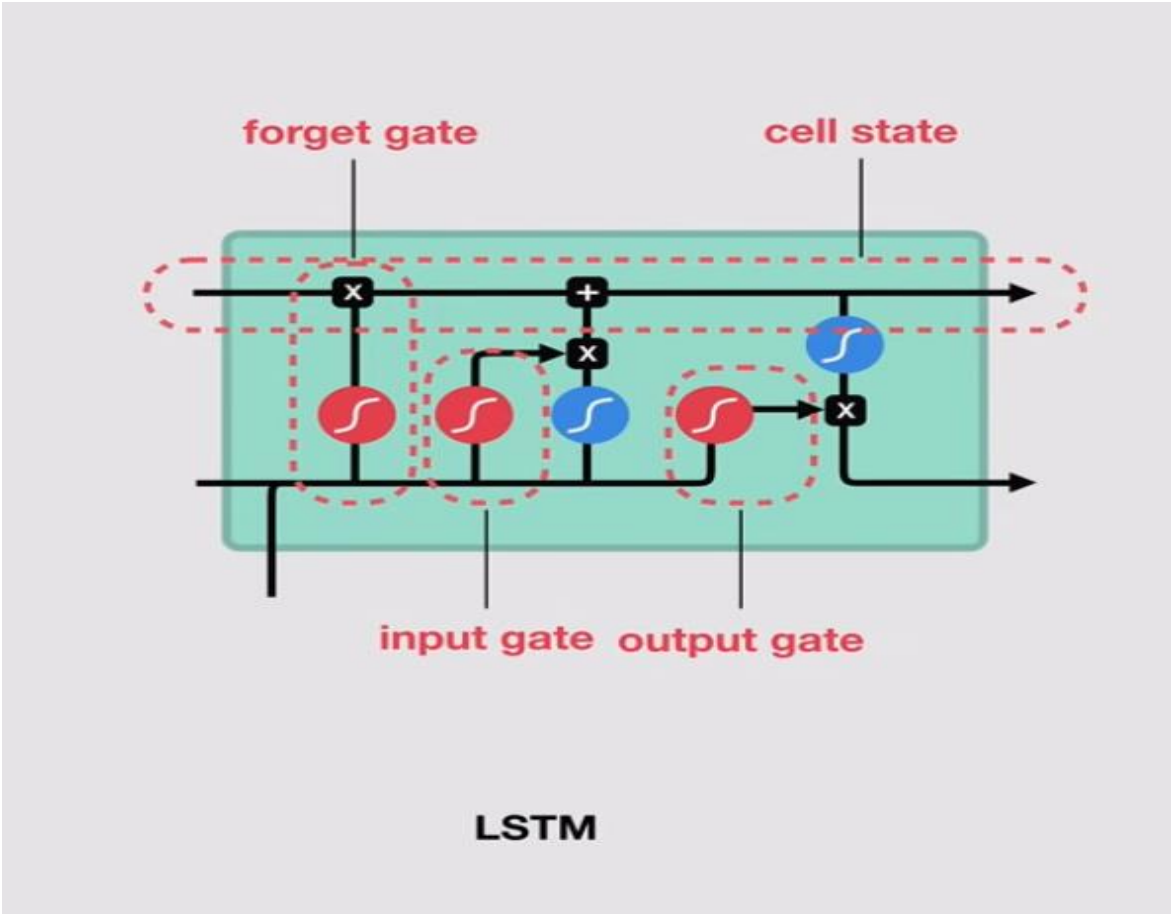
- The basic difference between the architectures of RNNs and LSTMs is that **the hidden layer of LSTM** is a gated unit or gated cell.
- The hidden layer of LSTM consists of **four layers** that interact with one another in a way to produce the output of that cell along with the cell state. These two things are then passed onto the next hidden layer.
- Unlike RNNs which have got the only **single neural net layer of tanh**, LSTMs comprises of **three logistic sigmoid gates and one tanh layer**.
- Gates have been introduced in order to limit the information that is passed through the cell. They determine which part of the information will be needed by the next cell and which part is to be discarded. The output is usually in the range of 0-1 where ‘0’ means ‘reject all’ and ‘1’ means ‘include all’.




These operations are used to allow the LSTM to keep or forget information.

# LSTM cells(LSTM Architecture)

- The core concept of LSTM's are the cell state, and it's various gates.
- The cell state is the "memory" of the network. The cell state can carry relevant information throughout the processing of the sequence. So information from the earlier time steps can make it's way to later time steps, reducing the effects of short-term memory. As the cell state goes on its journey, information get's added or removed to the cell state via gates. The gates are different neural networks that decide which information is allowed on the cell state. The gates can learn what information is relevant to keep or forget during training.



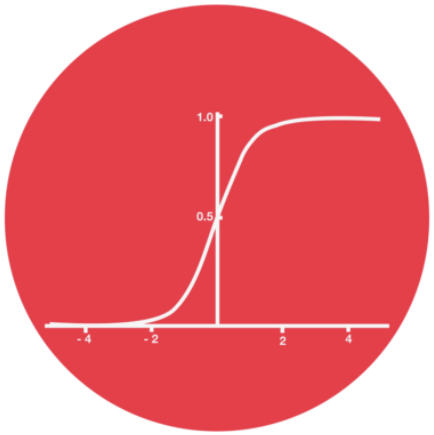

LSTM

# Sigmoid

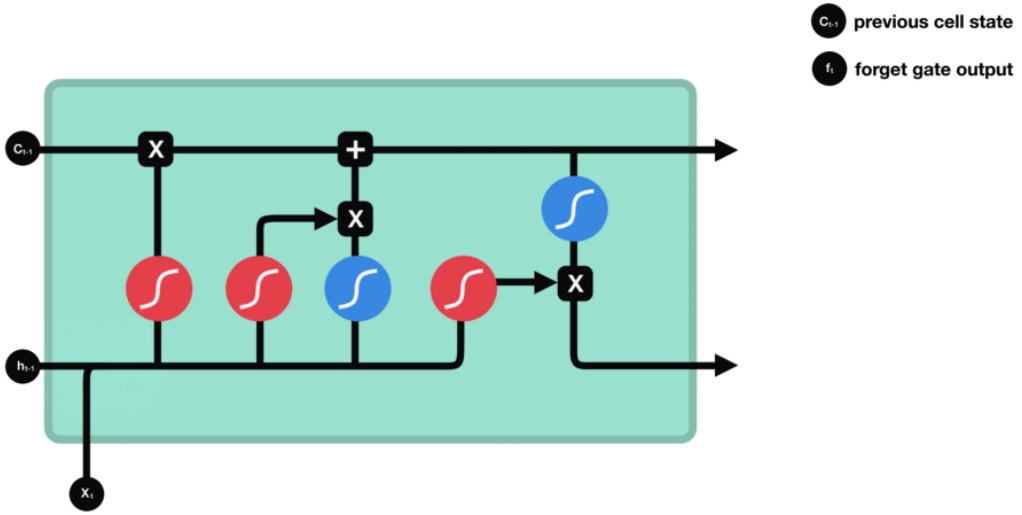
- Gates contains sigmoid activations.
- A sigmoid activation is similar to the tanh activation. Instead of squishing values between -1 and 1, it squishes values between 0 and 1. That is helpful to update or forget data because any number getting multiplied by 0 is 0, causing values to disappears or be “forgotten.” Any number multiplied by 1 is the same value therefore that value stay’s the same or is “kept.” Thus, the network can learn which data is not important therefore can be forgotten or which data is important to keep.

# Sigmoid

5
0.1
-0.5




# Forget Gate



- There are three different gates that regulate information flow in an LSTM cell. A forget gate, input gate, and output gate.

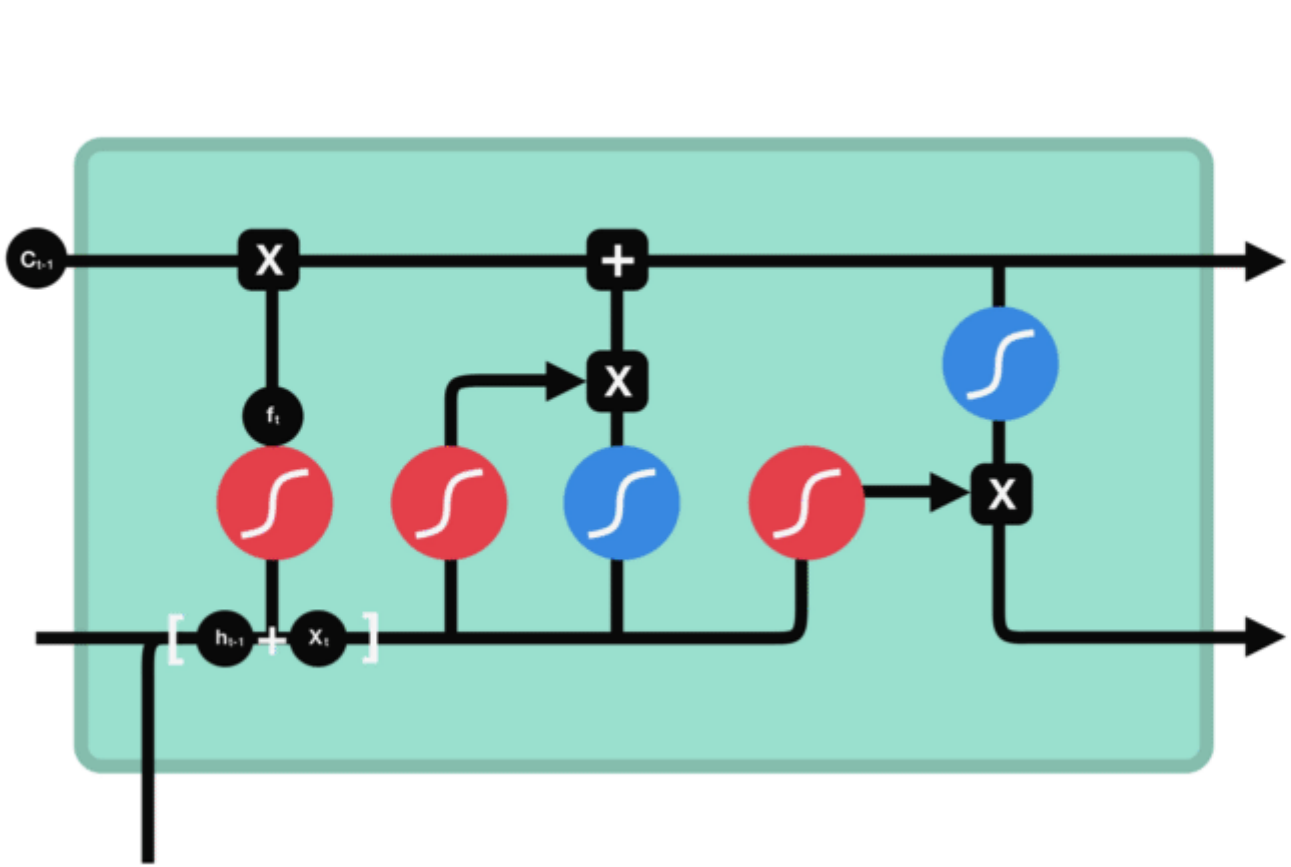
# Forget gate

- The forget gate decides what information should be thrown away or kept.
- Information from the previous hidden state and information from the current input is passed through the sigmoid function. Values come out between 0 and 1. The closer to 0 means to forget, and the closer to 1 means to keep.

# Input Gate

- To update the cell state, the input gate is used.
- First, pass the previous hidden state and current input into a sigmoid function. That decides which values will be updated by transforming the values to be between 0 and 1. 0 means not important, and 1 means important.
- Then pass the hidden state and current input into the tanh function to squish values between -1 and 1 to help regulate the network. Then multiply the tanh output with the sigmoid output. The sigmoid output will decide which information is important to keep from the tanh output.

# Input gate Operation




$C_{t-1}$ previous cell state

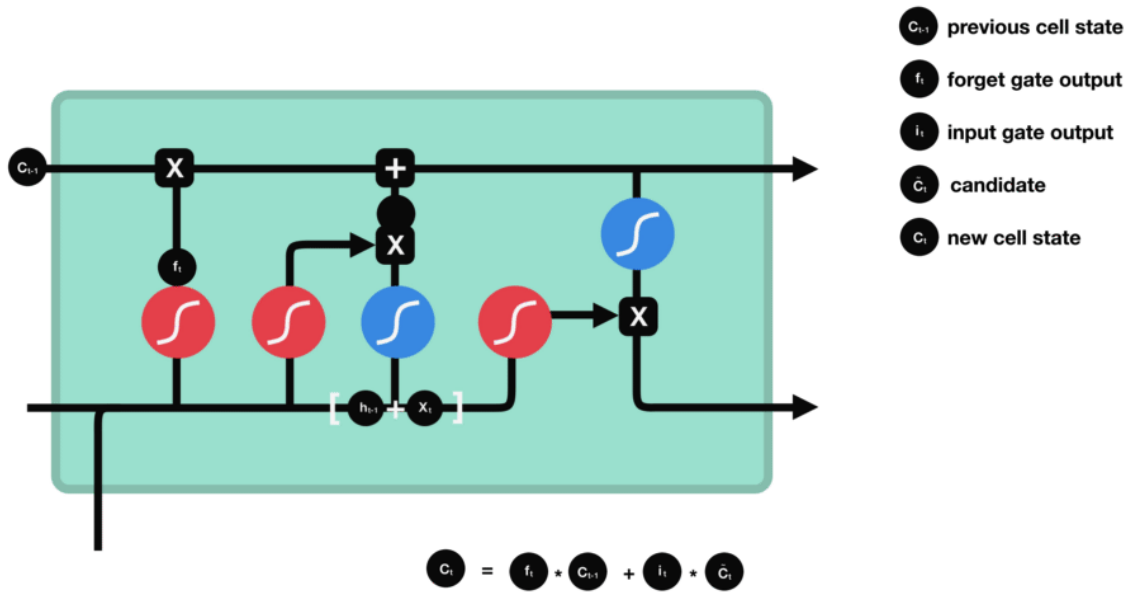
$f_t$ forget gate output

$i_t$ input gate output

$\tilde{C}_t$ candidate

# Cell State

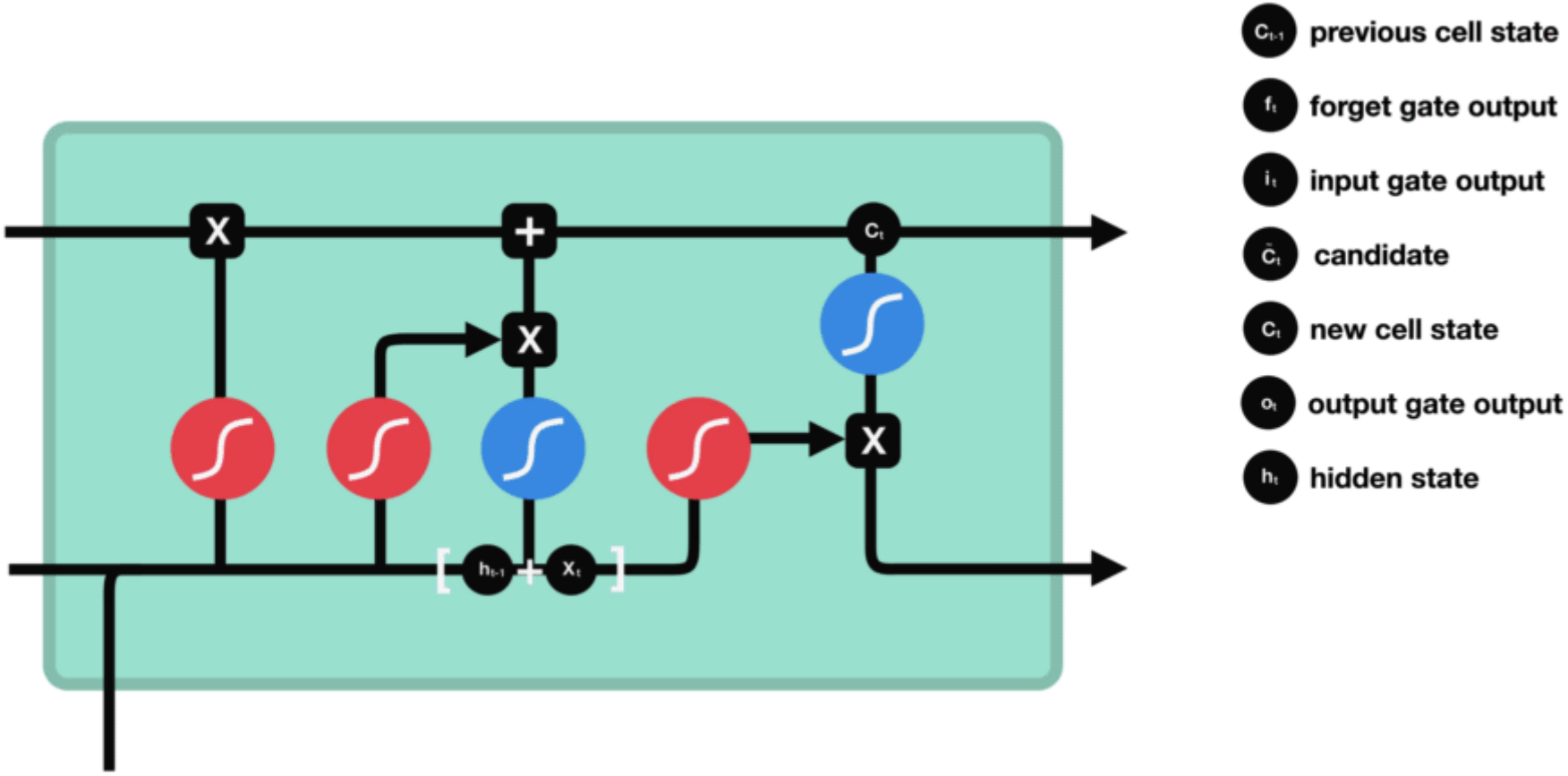
- First, the cell state gets pointwise multiplied by the forget vector. This has a possibility of dropping values in the cell state if it gets multiplied by values near 0. Then we take the output from the input gate and do a pointwise addition which updates the cell state to new values that the neural network finds relevant. That gives us our new cell state.



# Output Gate

- The output gate decides what should be the next hidden state.
- The hidden state contains information on previous inputs. The hidden state is also used for predictions.
- First, we pass the previous hidden state and the current input into a sigmoid function. Then we pass the newly modified cell state to the tanh function. Multiply the tanh output with the sigmoid output to decide what information the hidden state should carry. The output is the hidden state. The new cell state and the new hidden is then carried over to the next time step.

# Output Gate




Forget gate decides what is relevant to keep from prior steps. The input gate decides what information is relevant to add from the current step. The output gate determines what the next hidden state should be.

## Customers Review 2,491

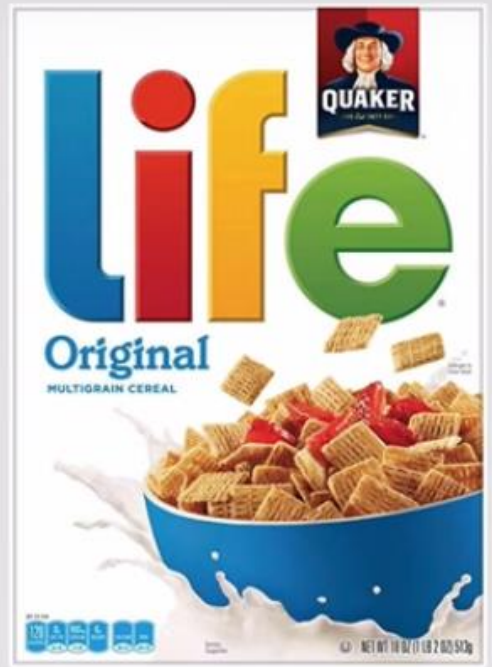


**Thanos**

September 2018

Verified Purchase

**Amazing! This box of cereal gave me a perfectly balanced breakfast, as all things should be. I only ate half of it but will definitely be buying again!**




**A Box of Cereal**

**$3.99**

## Customers Review 2,491

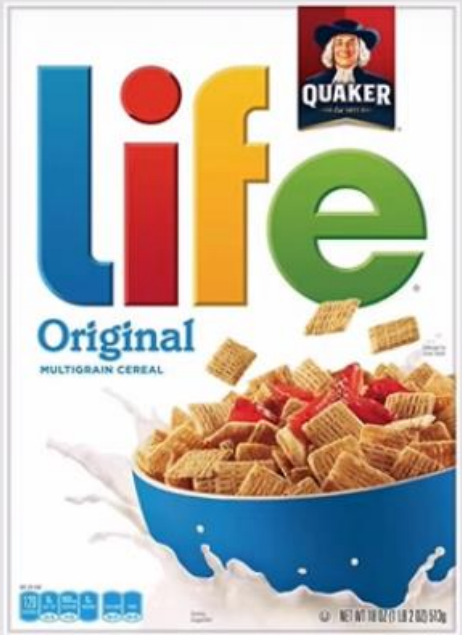


**Thanos**

September 2018

Verified Purchase

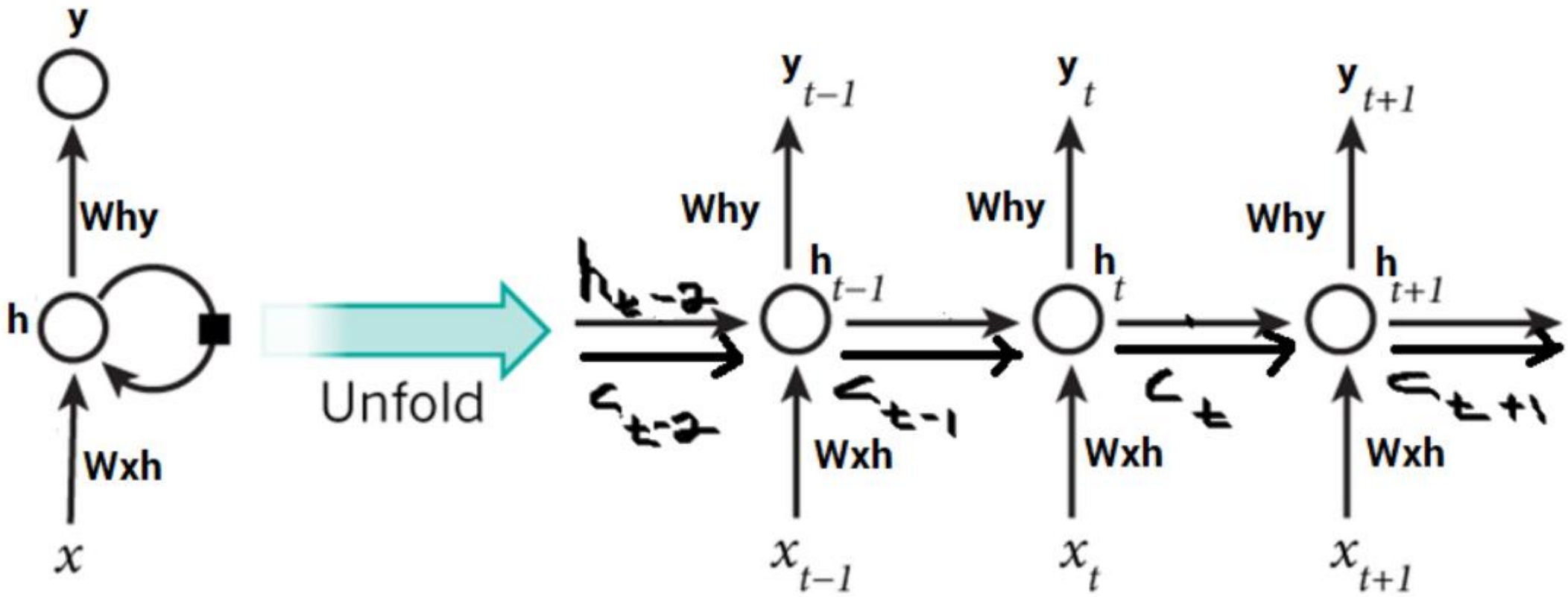
**Amazing!** This box of cereal gave me a **perfectly balanced breakfast**, as all things should be. I only ate half of it but **will definitely be buying again!**




**A Box of Cereal**

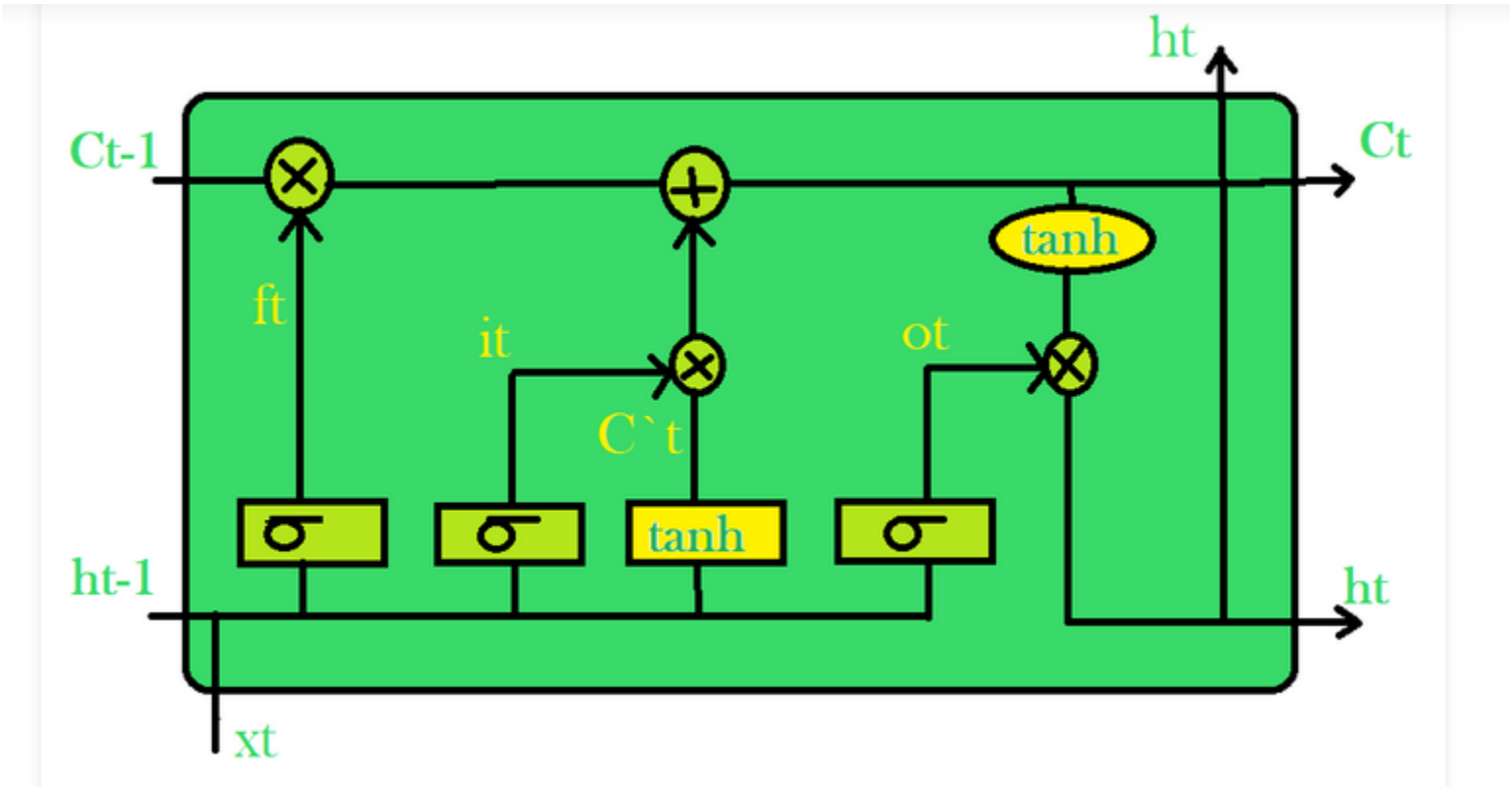
$3.99

# Hidden layer of LSTM



# How LSTM works

- Each LSTM cell has three inputs $h_{t-1}$ ,$C_{t-1}$ and $x_t$ and two outputs $h_t$ and $C_t$ . For a given time t, $h_t$ is the hidden state, $C_t$ is the cell state or memory, $x_t$ is the current data point or input. The **first sigmoid layer** has two inputs– $h_{t-1}$ and $x_t$ where $h_{t-1}$ is the hidden state of the previous cell. It is known as the **forget gate** as its output selects the amount of information of the previous cell to be included. The output is a number in [0,1] which is multiplied (point-wise) with the previous cell state $C_{t-1}$ .


ht
Ct-1
Ct
tanh
ft
it
ot
C`t
tanh
ht-1
ht
xt

- The **second sigmoid layer** is the **input gate** that decides what new information is to be added to the cell. It takes two inputs $h_{t-1}$ and $x_t$ . The tanh layer creates a vector $C_t$ of the new candidate values. Together, these two layers determine the information to be stored in the cell state. Their point-wise multiplication ( $i_t$ ? $C_t$ ) tells us the amount of information to be added to the cell state. The result is then added with the result of the forget gate multiplied with previous cell state ( $f_t$* $C_{t-1}$ ) to produce the current cell state $C_t$ .
- Next, the **output of the cell is calculated using a sigmoid and a tanh layer**. The sigmoid layer decides which part of the cell state will be present in the output whereas tanh layer shifts the output in the range of [-1,1]. The results of the two layers undergo point-wise multiplication to produce the output $h_t$ of the cell.

# LSTM Applications

1.Language modelling or text generation :

It involves the computation of words when a sequence of words is fed as input. Language models can be operated at the character level, sentence level or even paragraph level.

2. Image processing:

It involves performing analysis of a picture and concluding its result into a sentence. For this, it's required to have a dataset comprising of a good amount of pictures with their corresponding descriptive captions. A model that has already been trained is used to predict features of images present in the dataset. This is photo data. The dataset is then processed in such a way that only the words that are most suggestive are present in it. This is text data. Using these two types of data, we try to fit the model. The work of the model is to generate a descriptive sentence for the picture one word at a time by taking input words that were predicted previously by the model and also the image.

3. Speech and Handwriting Recognition

4. Music generation which is quite similar to that of text generation where LSTMs predict musical notes instead of text by analyzing a combination of given notes fed as input.

5. Language Translation involves mapping a sequence in one language to a sequence in another language. Similar to image processing, a dataset, containing phrases and their translations, is first cleaned and only a part of it is used to train the model. An encoder-decoder LSTM model is used which first converts input sequence to its vector representation (encoding) and then outputs it to its translated version.

# Drawbacks of LSTM

1. LSTMs became popular because they could solve the problem of vanishing gradients. But they failed to remove it completely.
2. They require a lot of resources and time to get trained and become ready for real-world applications. LSTM need high memory-bandwidth because of linear layers present in each cell which the system usually fails to provide for. Thus, hardware-wise, LSTMs become quite inefficient.
3. With the rise of data mining, developers are looking for a model that can remember past information for a longer time than LSTMs.
4. LSTMs get affected by different random weight initialization and hence behave quite similar to that of a feed-forward neural net. They prefer small weight initialization instead.

5. LSTMs are prone to overfitting and it is difficult to apply the dropout algorithm to curb this issue. Dropout is a regularization method where input and recurrent connections to LSTM units are probabilistically excluded from activation and weight updates while training a network.

# GRU

- The GRU is the newer generation of Recurrent Neural networks and is similar to an LSTM.
- GRU's got rid of the cell state and used the hidden state to transfer information.
- It also only has two gates, a reset gate and update gate.

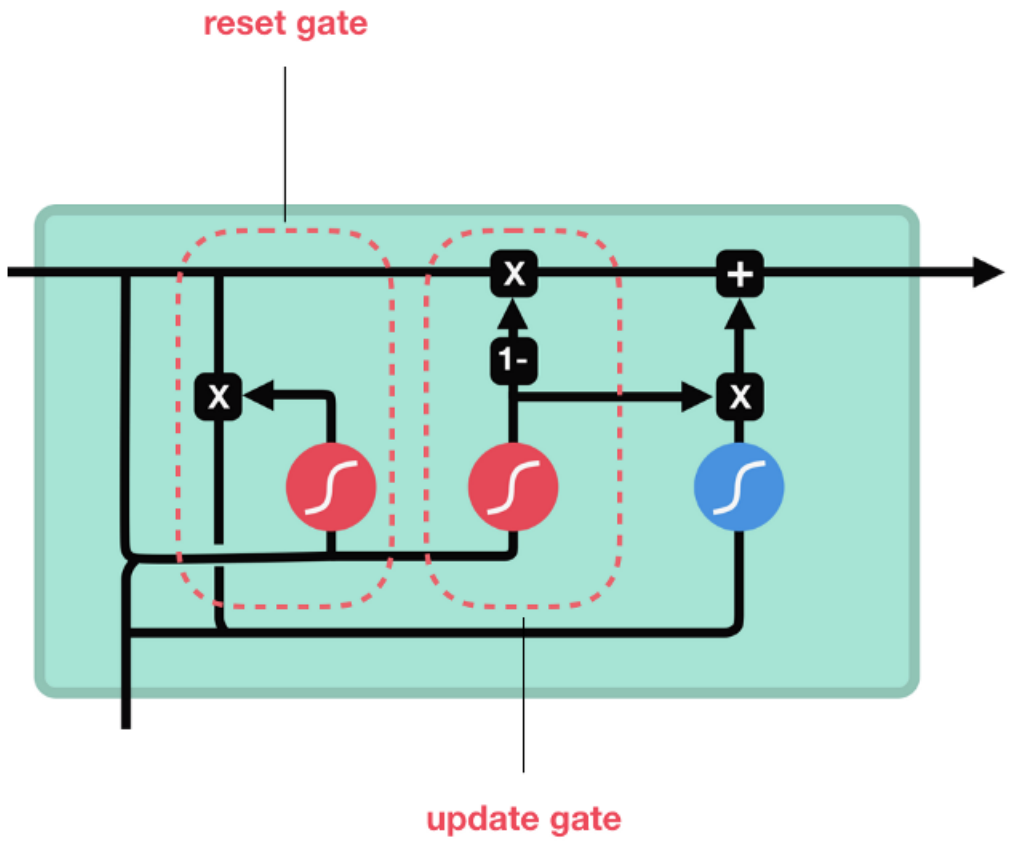
## Update Gate

- The update gate acts similar to the forget and input gate of an LSTM. It decides what information to throw away and what new information to add.

## Reset Gate

- The reset gate is another gate is used to decide how much past information to forget.
- GRU's has fewer tensor operations; therefore, they are a little speedier to train then LSTM's.

# GRU cell and it’s states



# How Long Short-Term Memory Networks solve Exploding or vanishing gradient problem

- Explain LSTM cells and how it works.

LSTM  prevents error gradients from vanishing

- RNNs suffer from vanishing gradients and are caused by long series of multiplications of small values, diminishing the gradients and causing the learning process to become degenerate. RNNs suffer from exploding gradients also which is affected from large gradient values , hampering the learning process.
- LSTMs solve the problem using a unique additive gradient structure that includes direct access to the forget gate’s activations, enabling the network to encourage desired behaviour from the error gradient using frequent gates update on every time step of the learning process.